Mount Kenya University

SCHOOL OF COMPUTING AND INFORMATICS DEPARTMENT

OF INFORMATION TECHNOLOGY

PROJECT TITLE: ART STUDIO

BY

NAME: JANE WANGUI GICHUHI

REG NO: BSCCS/2024/32659

UNIT CODE: BIT3208

UNIT TITLE: ADVANCED WEB AND DEVELOPMENT

LECTURER NAME: MICHAEL NYORO

SEM: YEAR 4 SEM 2

TECHNOLOGIES USED

MongoDB	React
Typescript	Vite GitHub
Vercel	PHP
Clerk	XAMPP
MySQL	

GitHub link:https://github.com/kuijennie/art-studio/tree/main

TABLE OF CONTENTS

WEEK 1: INSTALLATION AND TESTING OF LOCAL DEVELOPMENT ENVIRONMENT

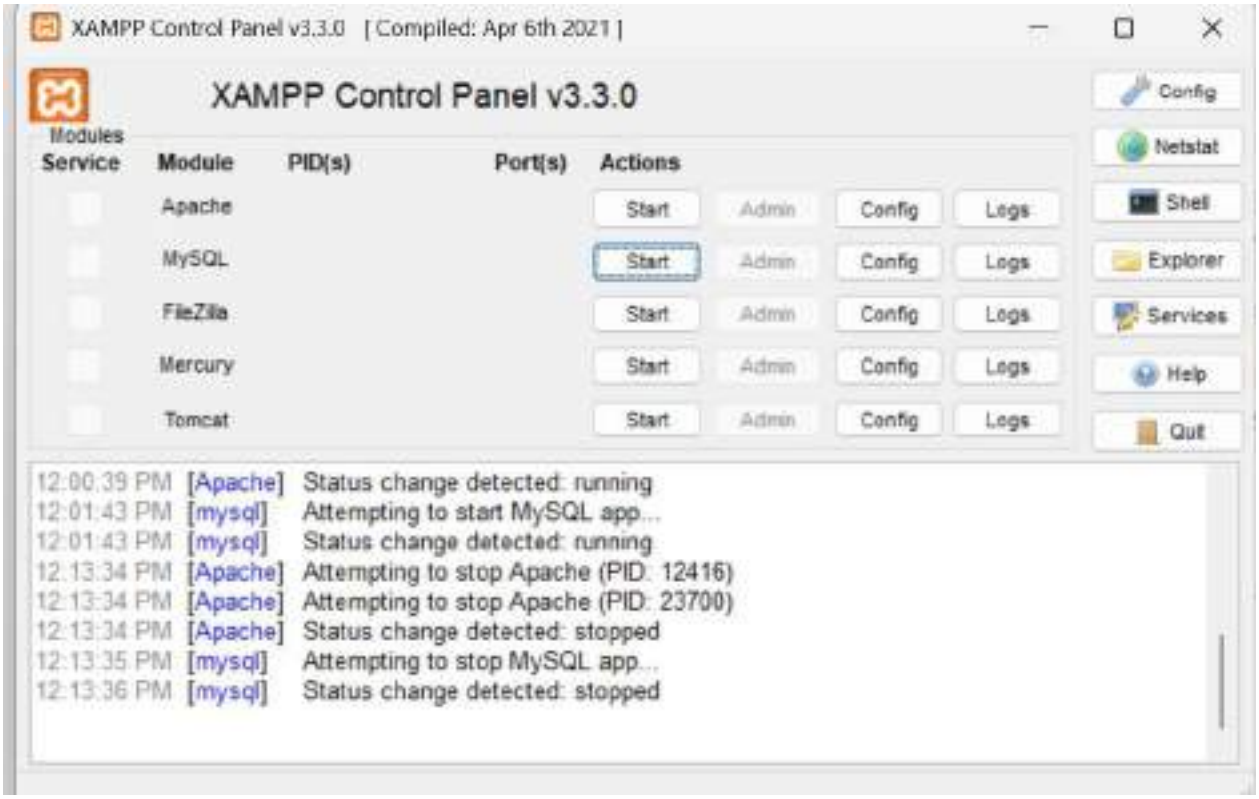

Fig 1: Shows the installation of XAMPP, a local server environment for running PHP applications.

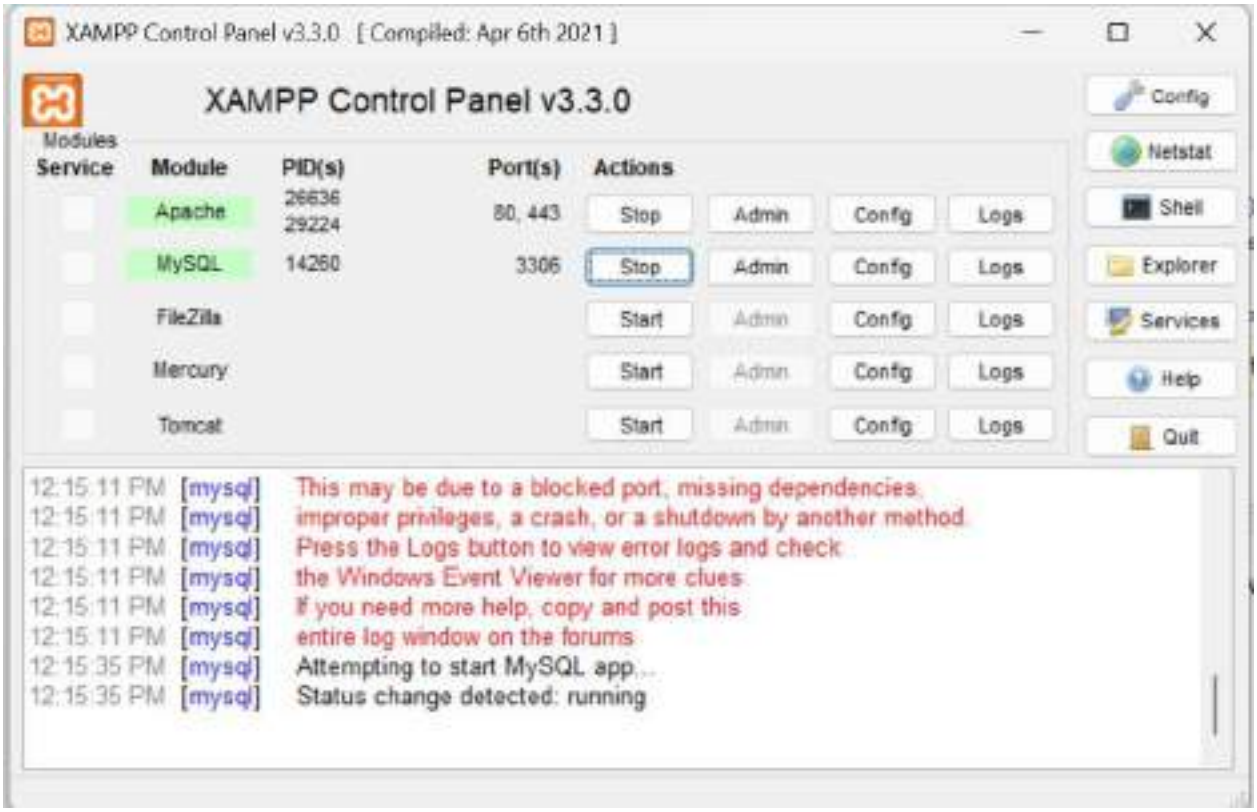

Fig 2: Shows Apache and MySQL services running on XAMPP, confirming the local server is active.

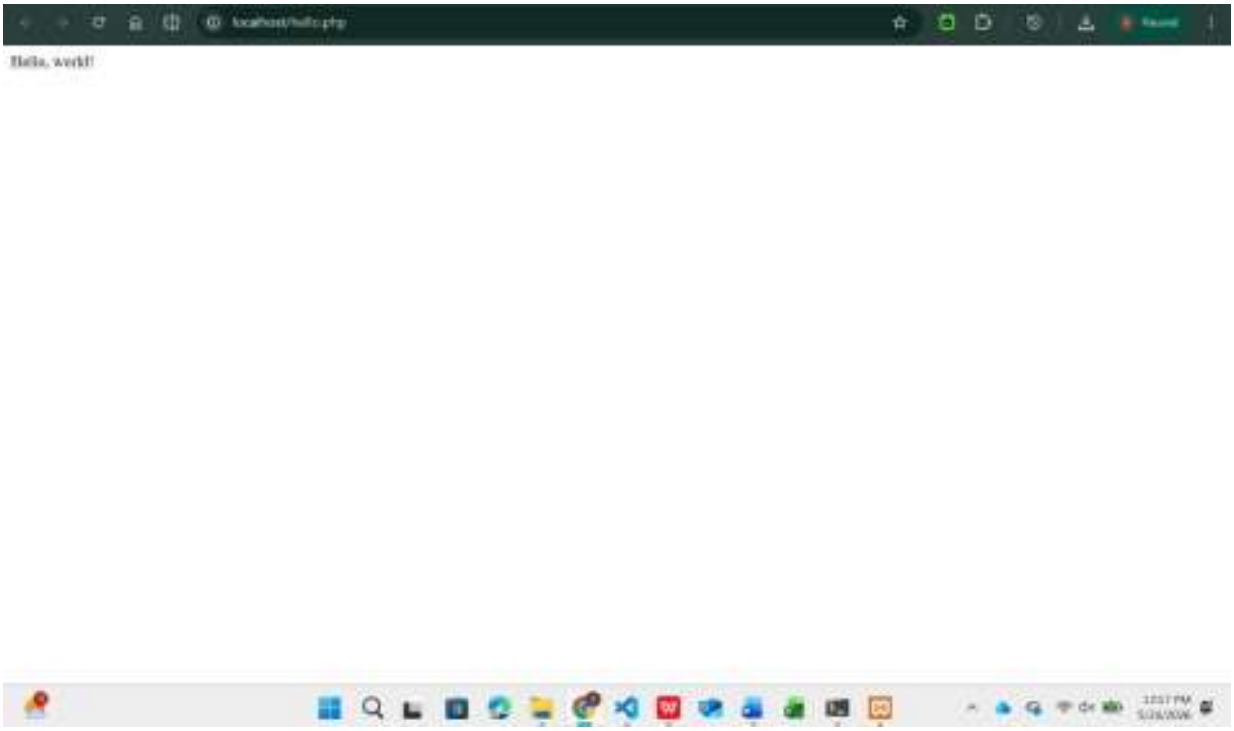

Fig 3: Localhost Test Page showing its working.

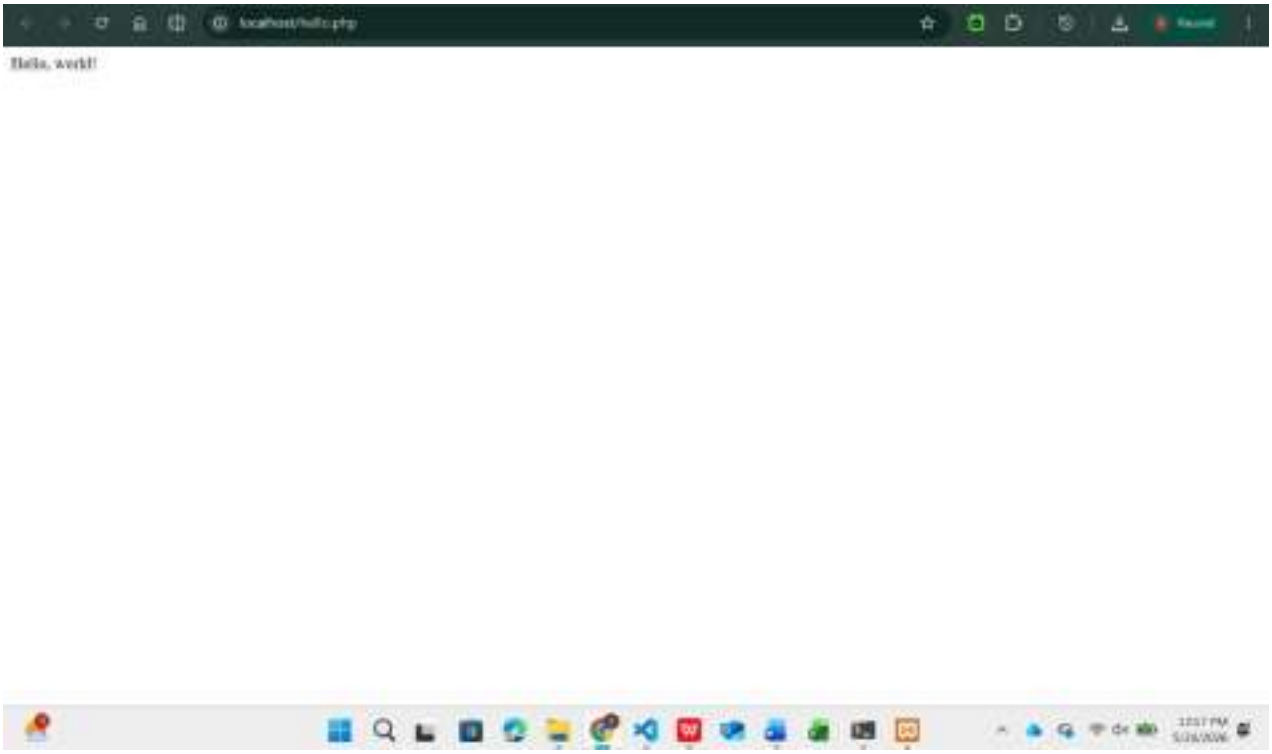

Fig 4: Displays a "Hello World" output, confirming PHP is correctly configured and executing.

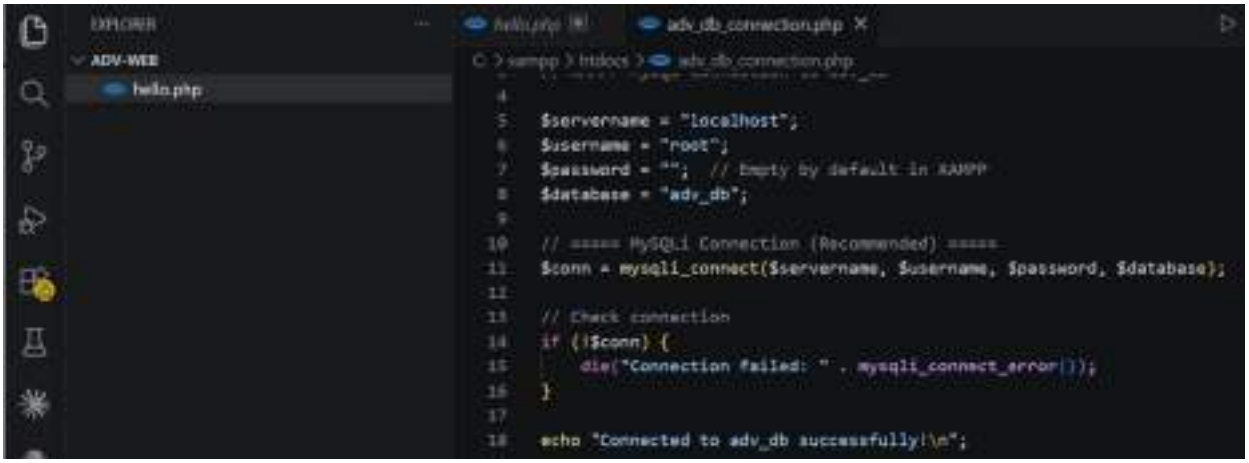

Fig 5a: Shows the initial database connection test to verify connectivity between PHP and MySQL.

Fig 5b: Shows the database connection test result, confirming a successful connection to the MySQL database.

My Reflection

I successfully set up a local development environment using XAMPP. Apache and MySQL were configured and run together, allowing PHP to communicate with the database. I also tested localhost, created a PHP “Hello World” page, and confirmed a successful PHP-MySQL connection using mysqli_connect(). The main challenge was fixing an Apache port conflict.

WEEK 2: USER INTERFACE PLANNING AND SYSTEM DESIGN

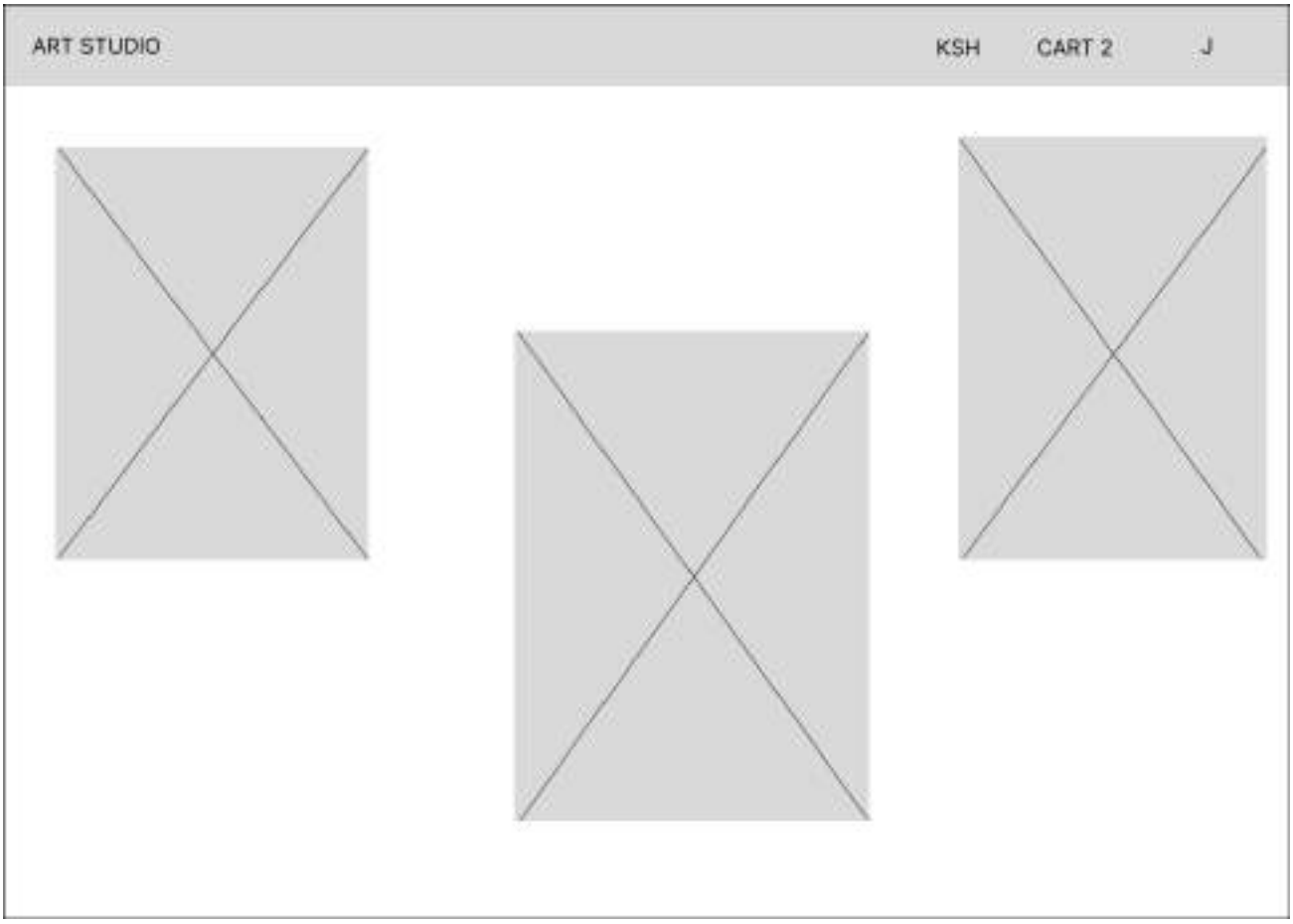

Fig 1a: Shows the wireframe design of the homepage layout, outlining the navigation bar and image placeholder arrangement

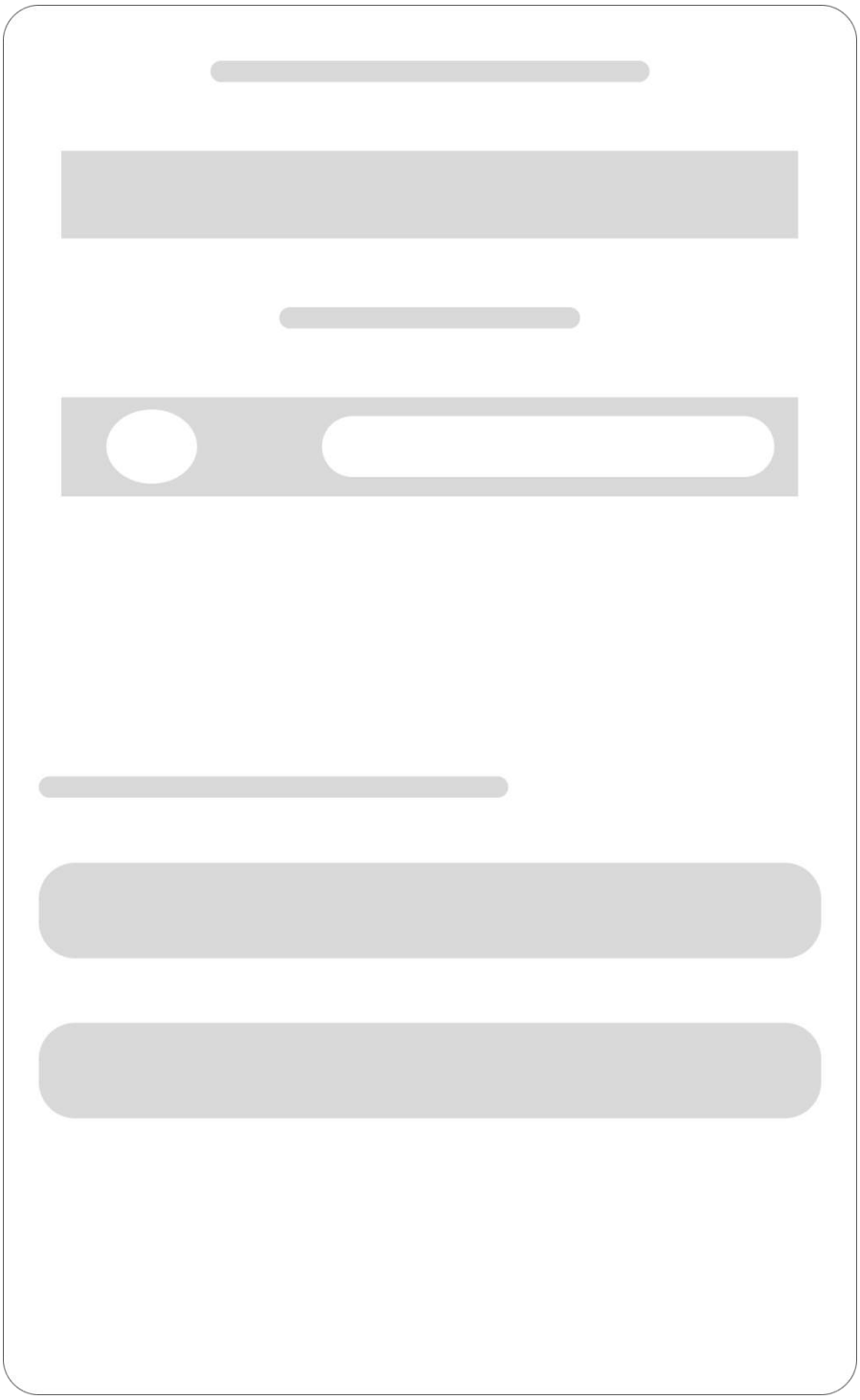

Fig 1b: Shows the wireframe design of the login page, outlining the input fields and authentication interface.

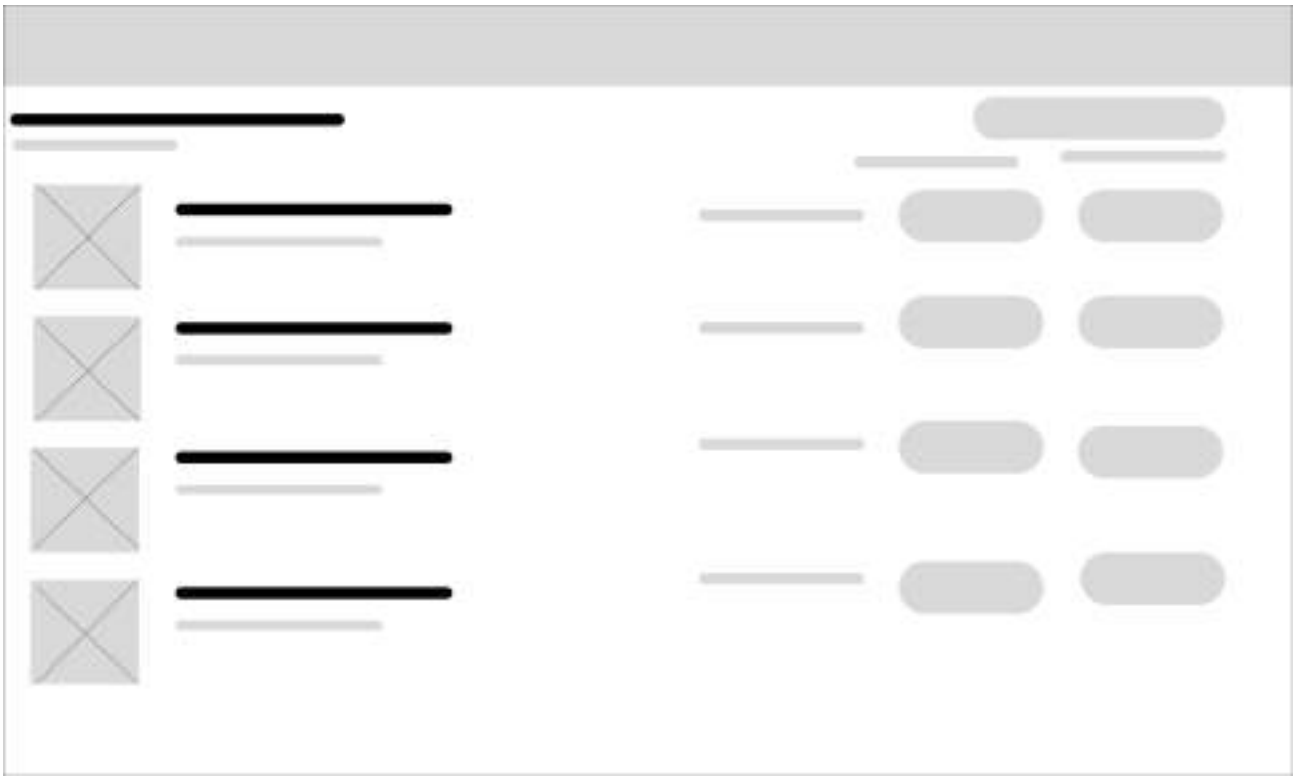

Fig 2: Illustrates the dashboard layout design, showing the arrangement of key UI components and navigation elements.

Fig 3: Displays the application's color theme, showing the primary, accent, and UI colors used consistently across the system.

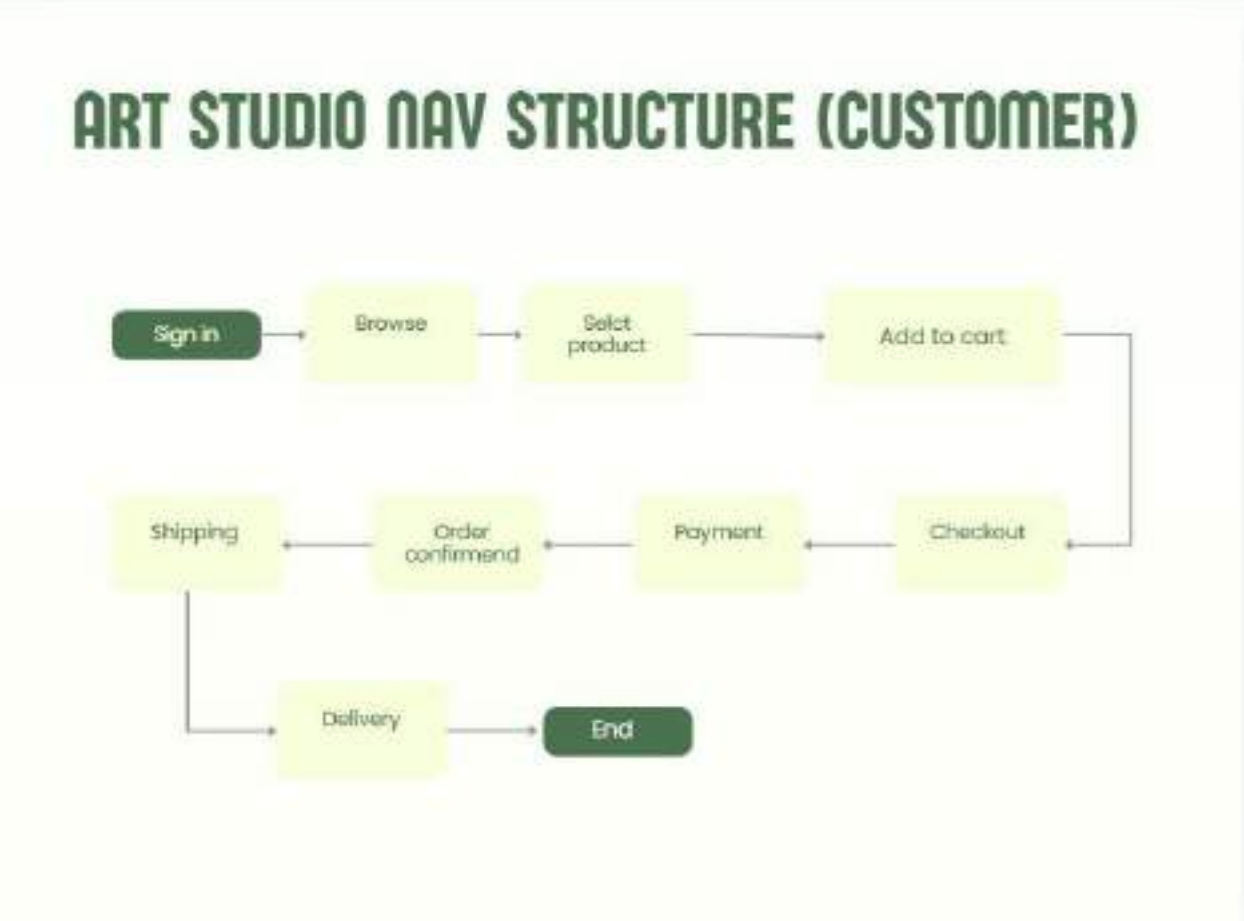

Fig 4a: Illustrates the customer navigation structure, showing the flow between pages available to a regular user.

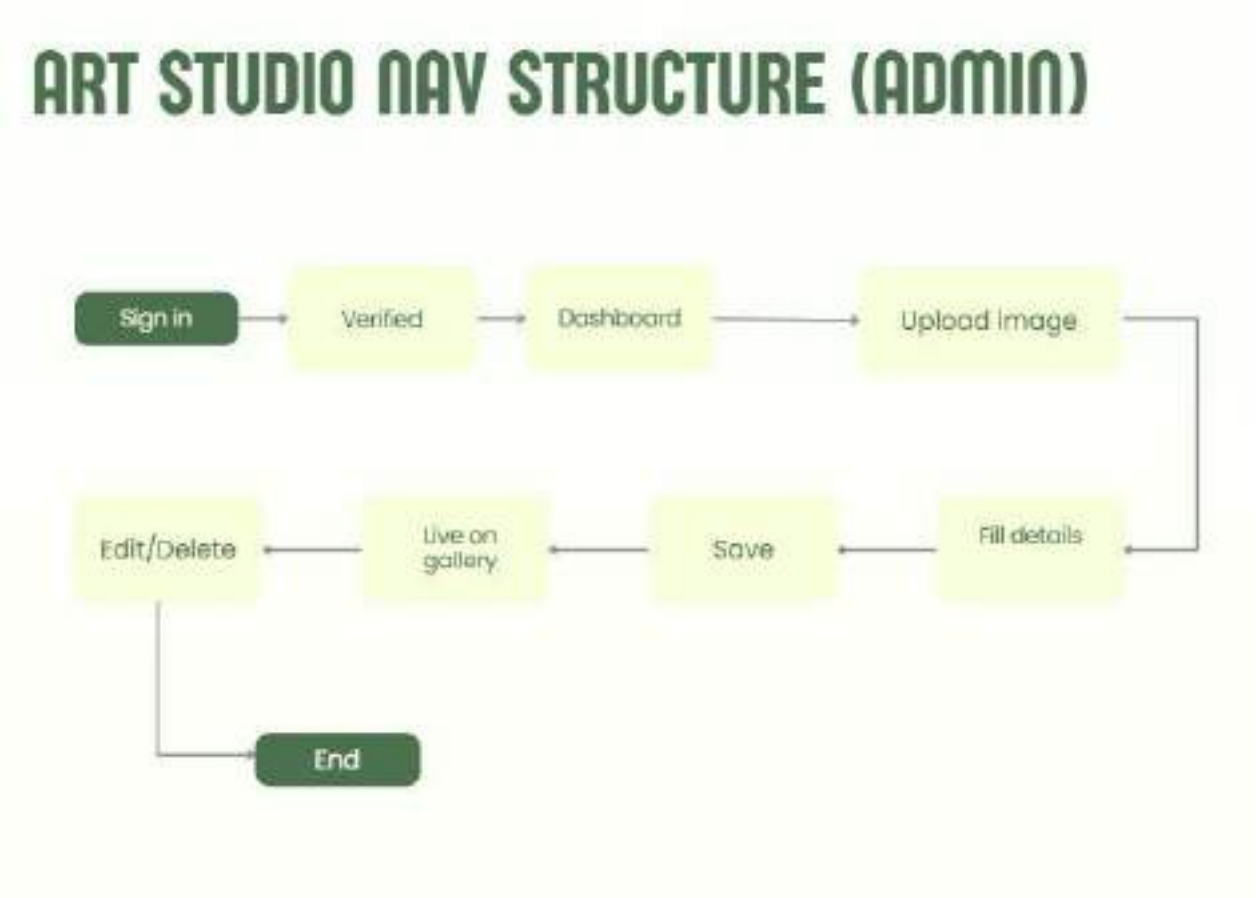

Fig 4b: Illustrates the admin navigation structure, showing the flow between pages and management functions available to an administrator.

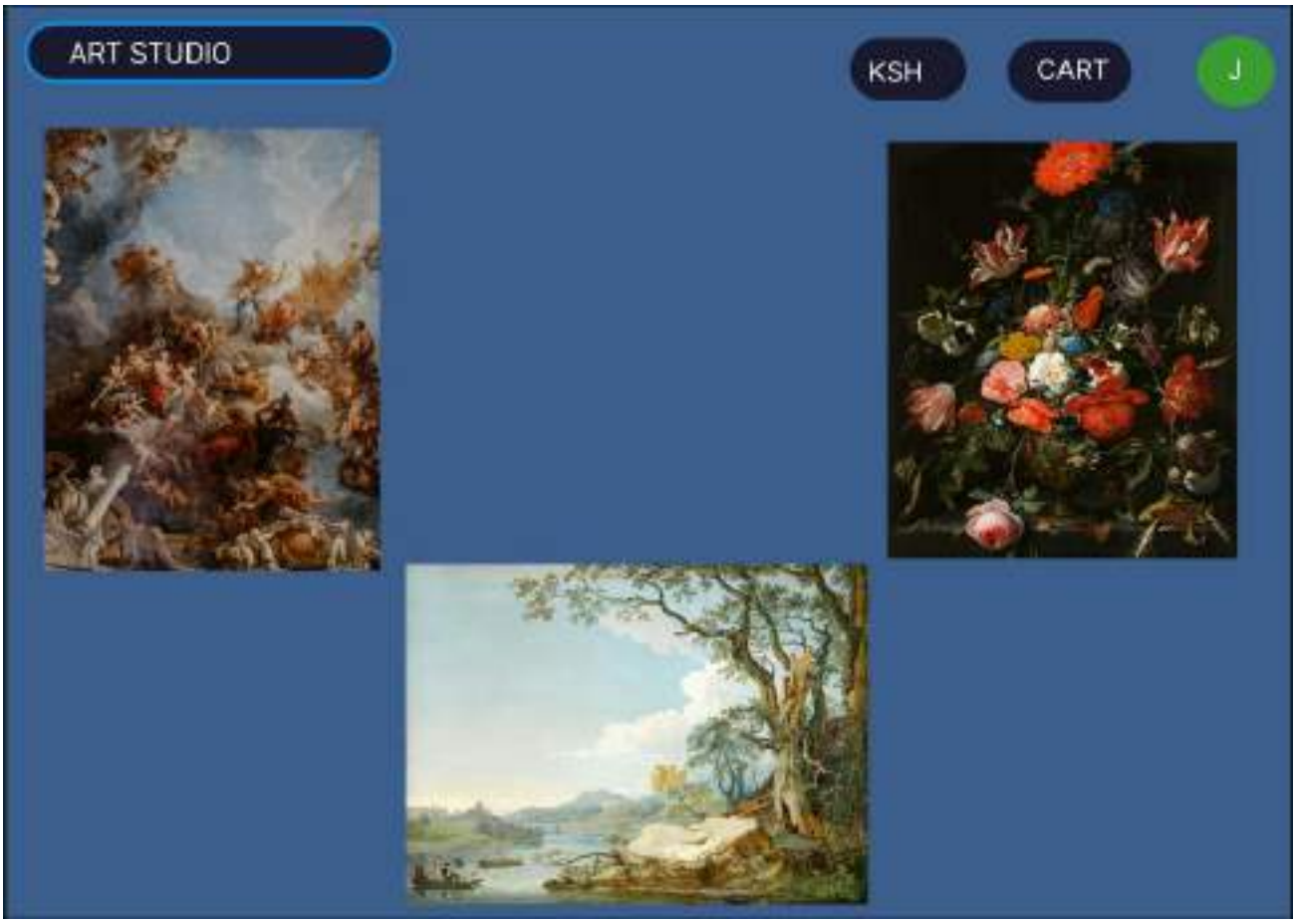

Fig 5a: Home page

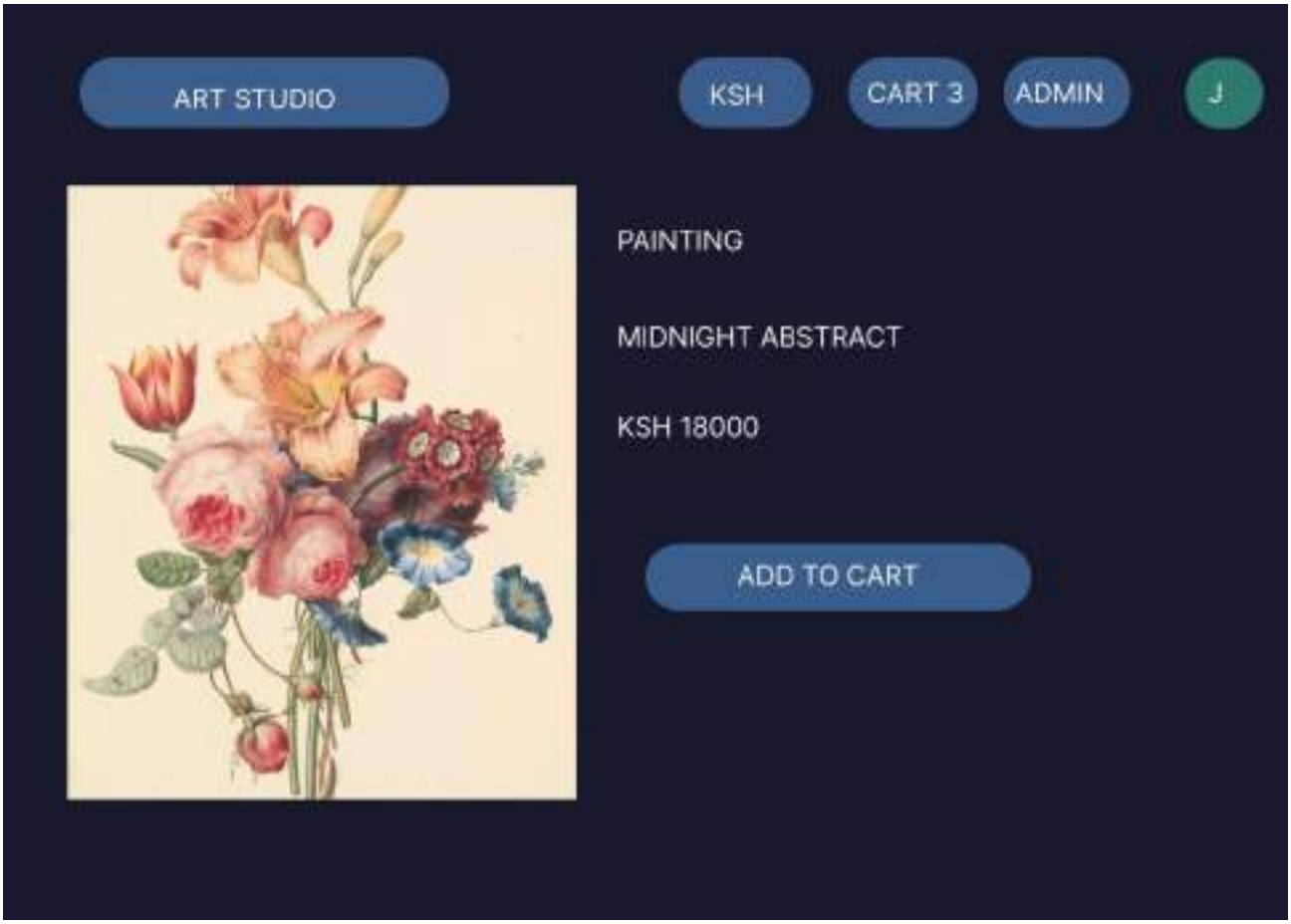

Fig 5b: Add to cart page

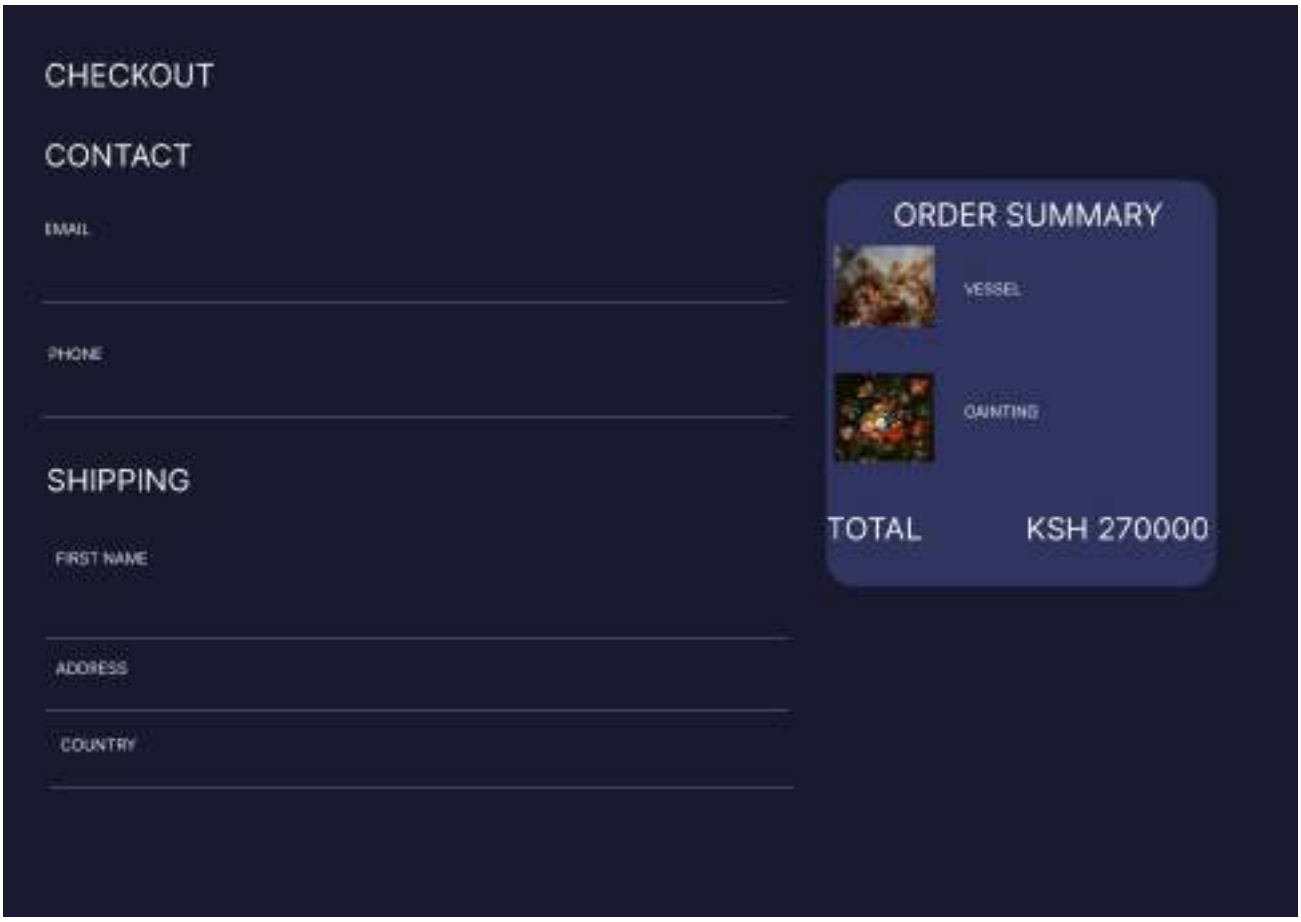

Fig 5c: Checkout page

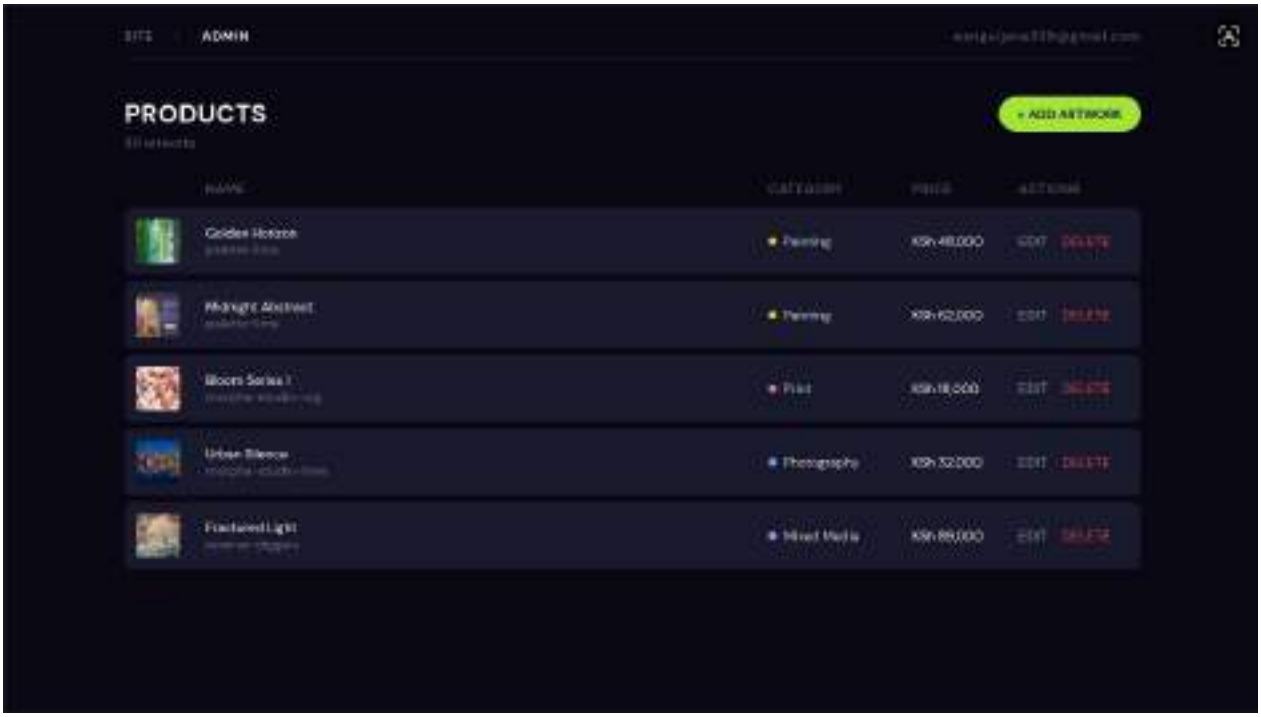

Fig 5d: Admin page

My Reflection

The Art Studio interface was designed to feel like a physical art museum, allowing the artwork to remain the main focus. A dynamic color theme, simple navigation, and clear typography improved the user experience. The system supports art enthusiasts purchasing artwork and admins managing products and orders efficiently.

WEEK 3: FRONTEND INTERACTION AND BACKEND FOUNDATIONS

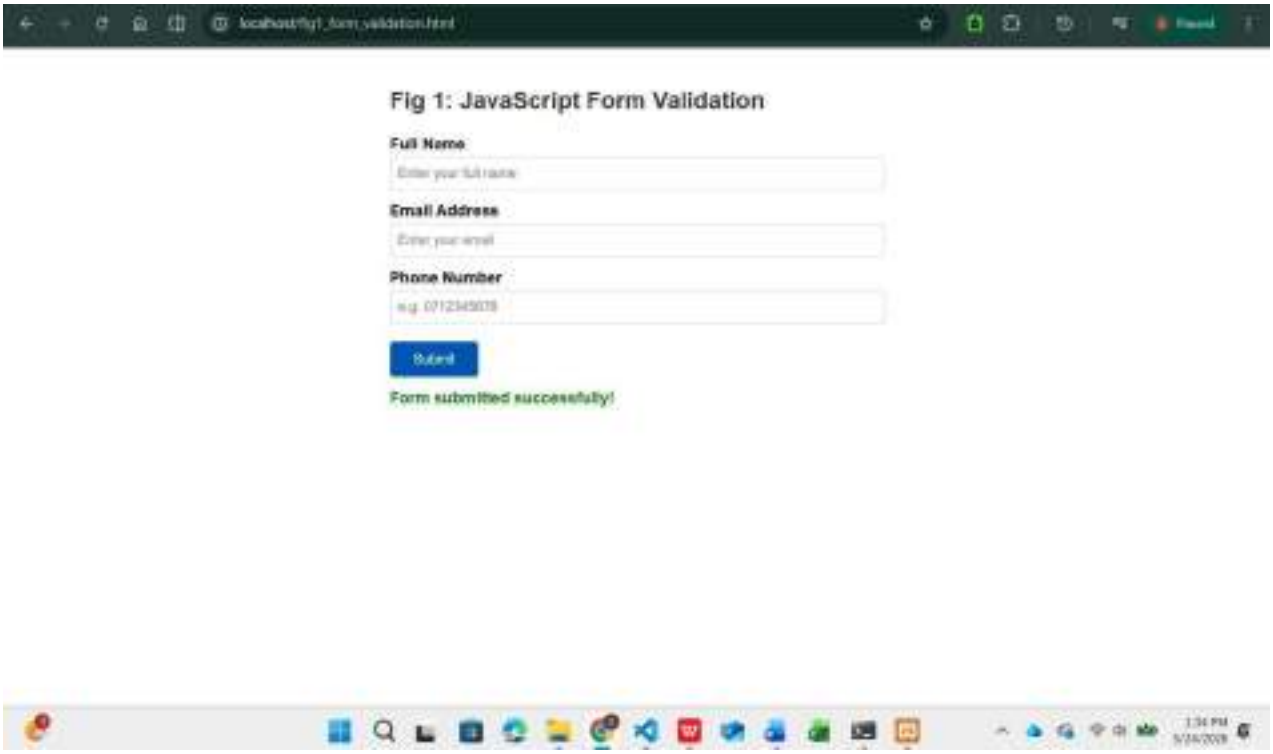

Fig 1a: Shows a JavaScript form validation script ensuring all input fields meet the required format before submission.

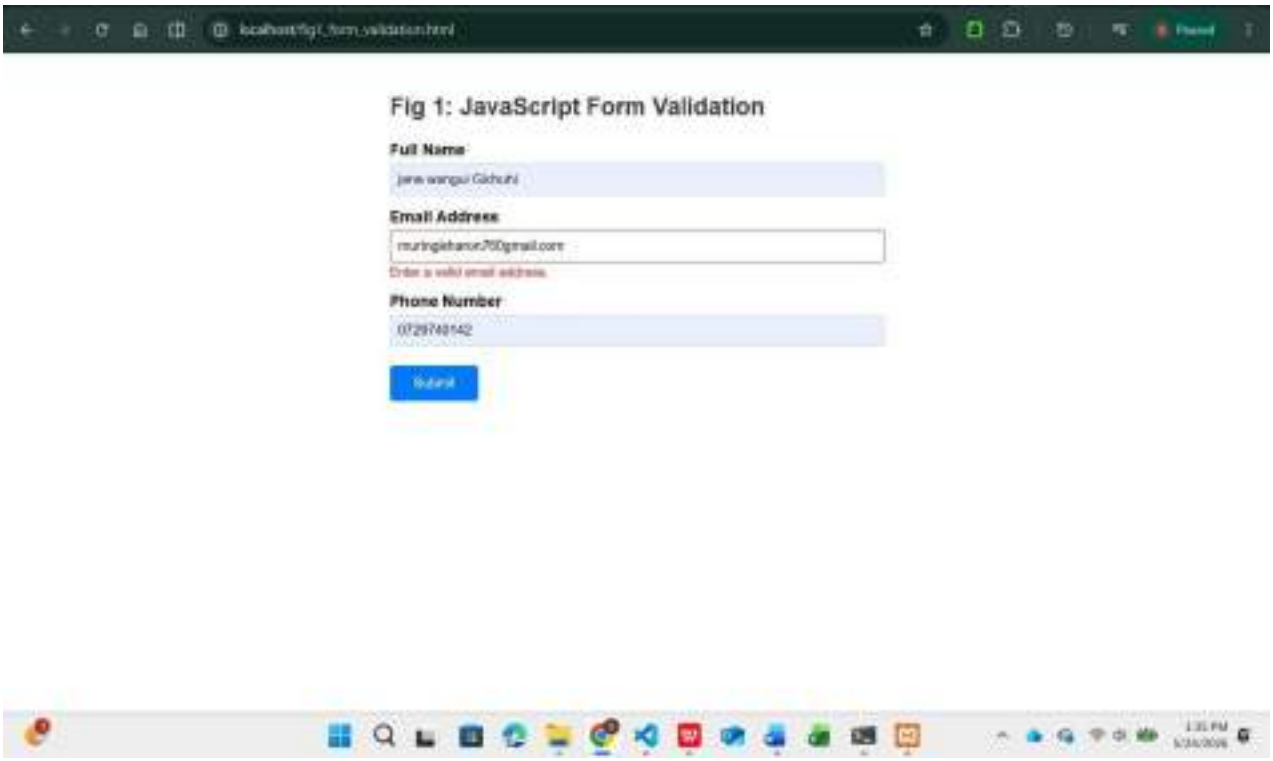

Fig 1b: Shows a JavaScript form validation script with incorrect input fields that do not meet the required format before submission.

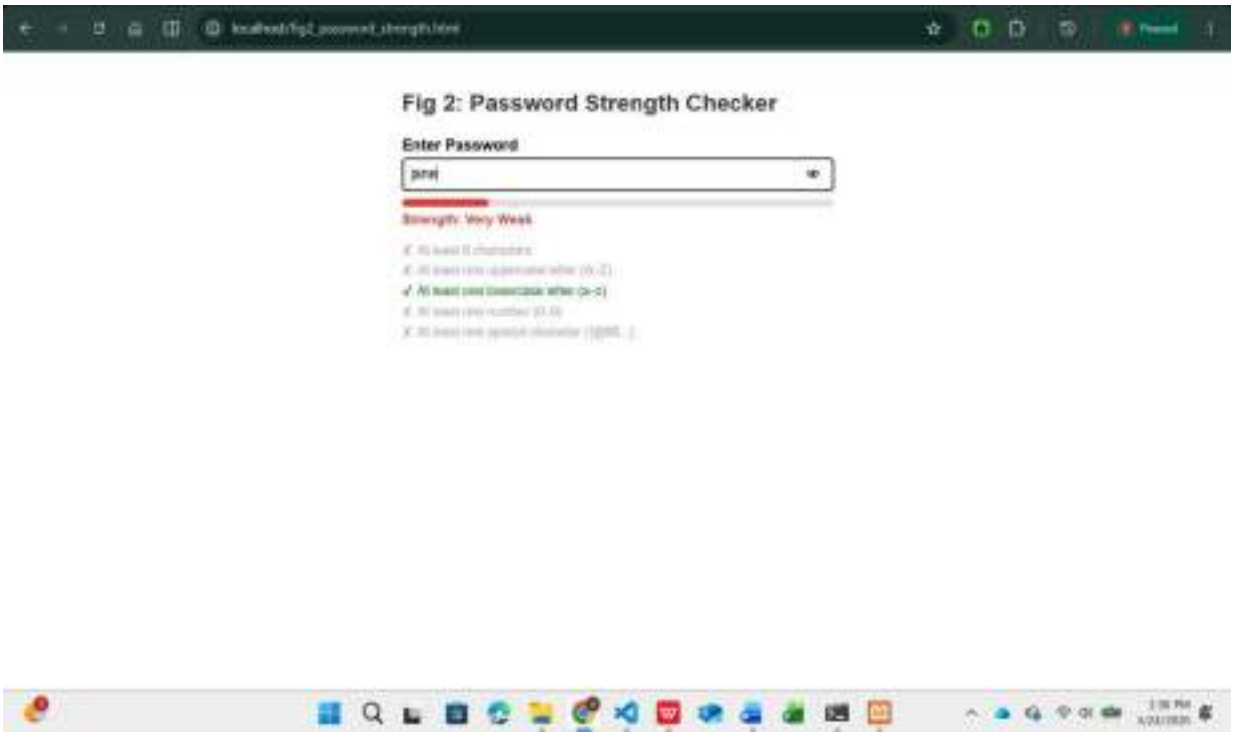

Fig 2a Displays a password strength checker that evaluates and indicates the security level of a user-entered password which is weak.

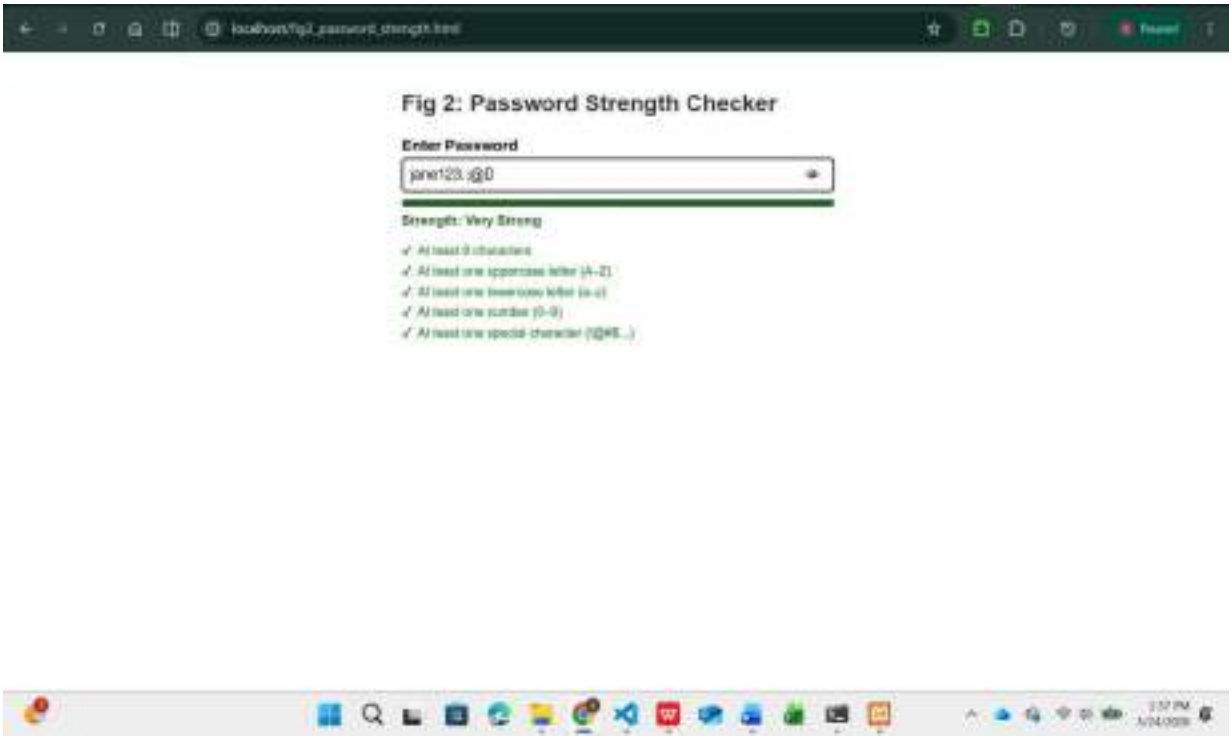

Fig 2b Displays a password strength checker that evaluates and indicates the security level of a user-entered password which is strong.

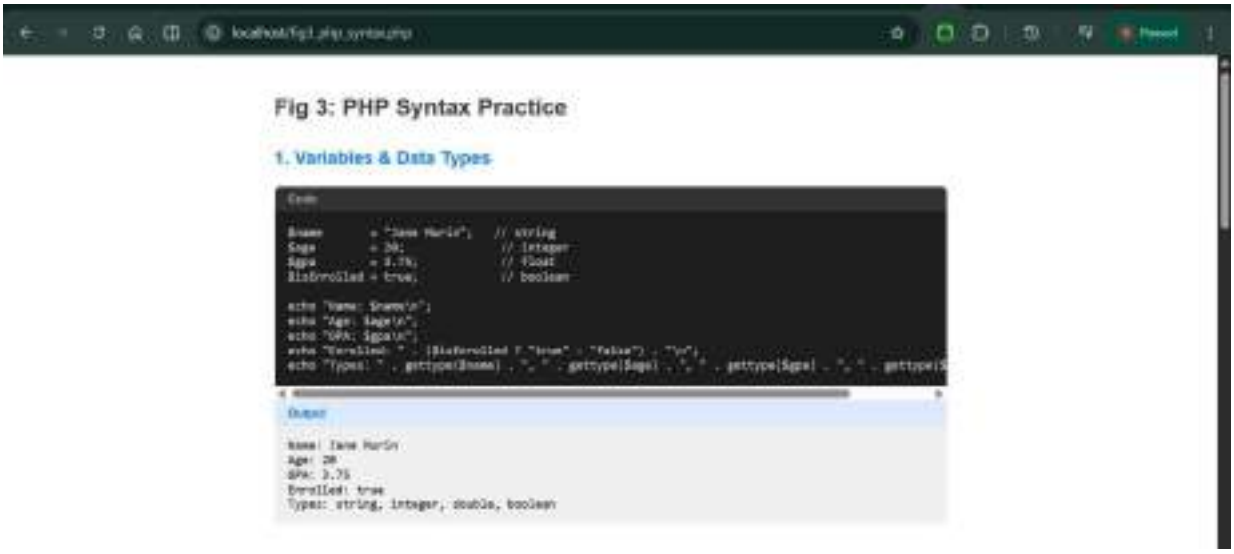

Fig 3: Illustrates PHP syntax practice, demonstrating correct usage of PHP structure, variables, and basic operation

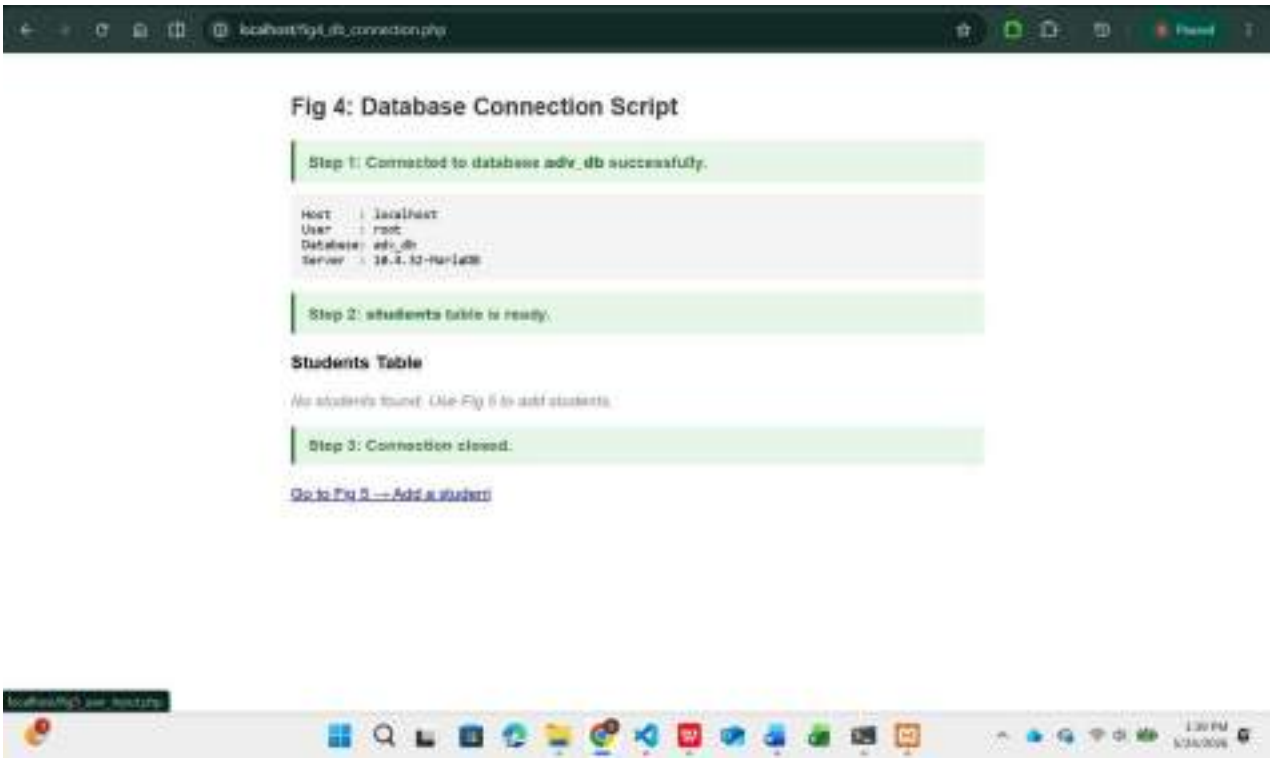

Fig 4: Shows a database connection script establishing a successful link between PHP and a MySQL database.

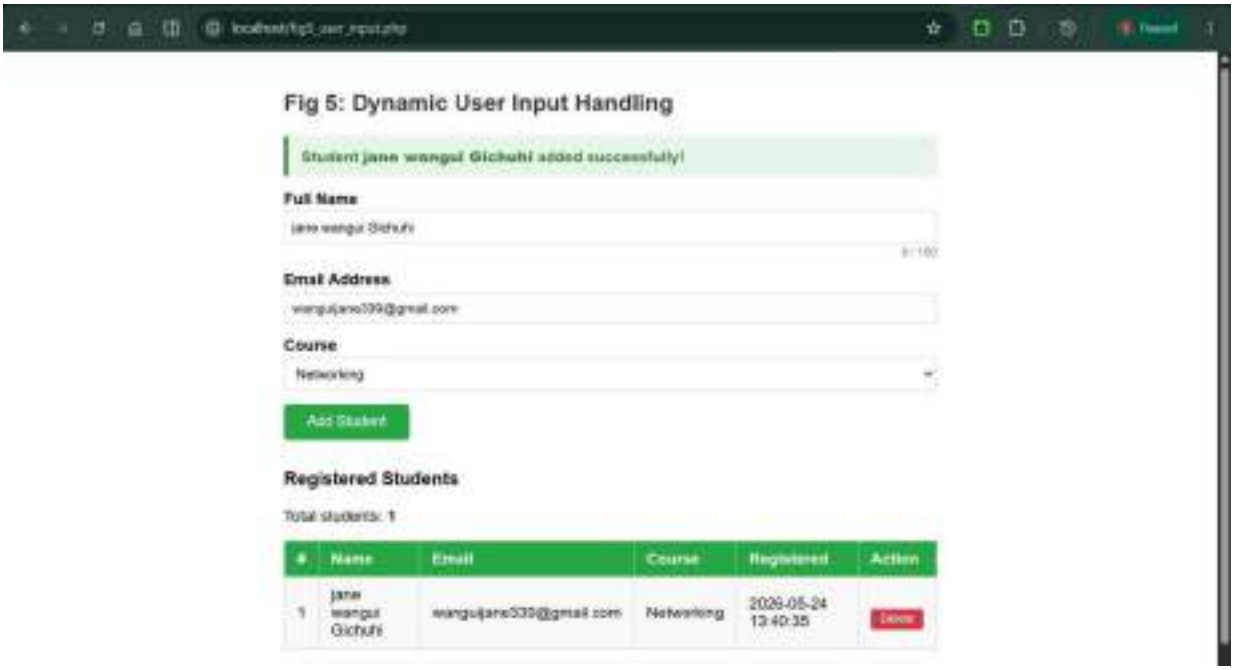

Fig 5: Demonstrates dynamic user input handling, where PHP processes and displays user-submitted data in real time.

My Reflection

I focused on core web scripting using JavaScript and PHP. Form validation improved user experience by giving instant feedback, while a password strength checker encouraged secure credentials. PHP practice strengthened server-side programming, and the database connection script supported dynamic data storage and retrieval. These exercises improved my ability to build secure and interactive web applications.

WEEK 4: SERVER-SIDE COMPONENTS AND BACKEND DEVELOPMENT FOUNDATIONS

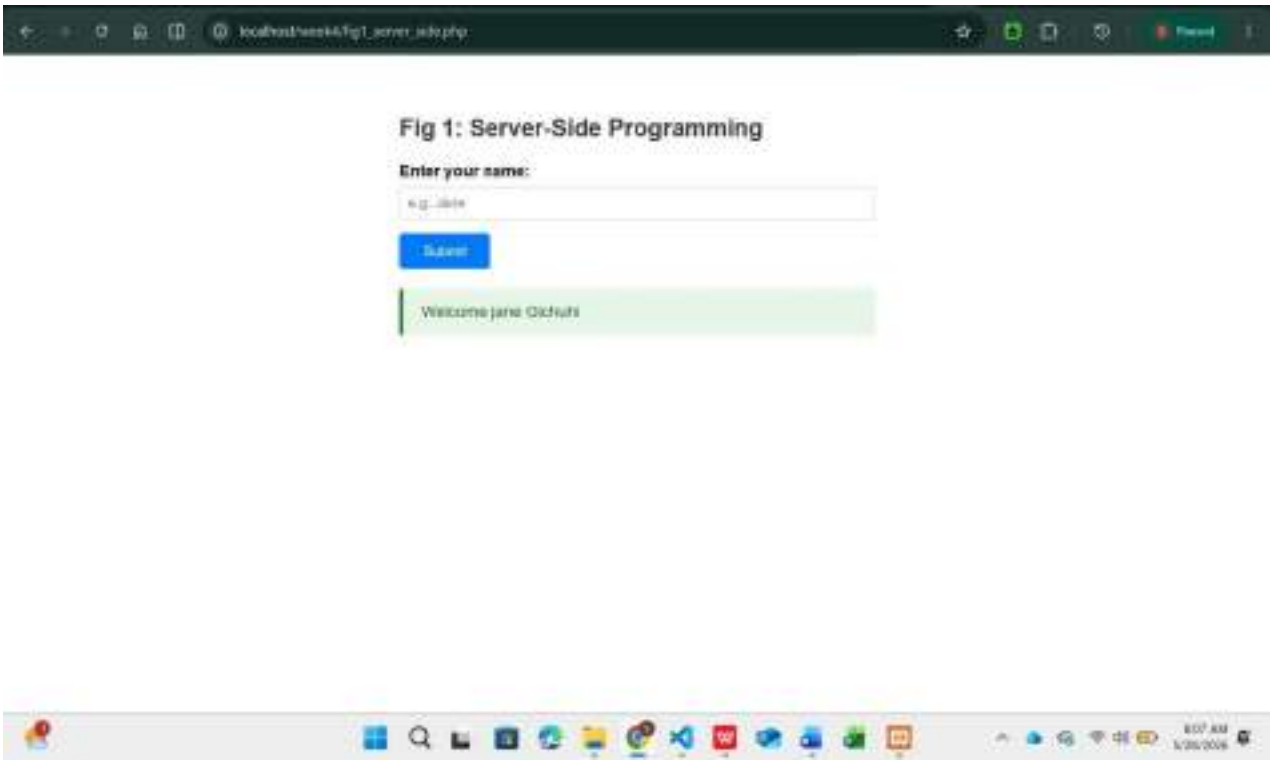

Fig 1: Shows a server-side PHP script processing a user-submitted name input and dynamically displaying a personalized welcome message.

Fig 2: Shows the HTML Forms and PHP Registration Form with a successful registration confirmation message for the user.

Fig 3: Displays the HTML Forms and PHP Login Form showing a successful login confirmation for the admin user.

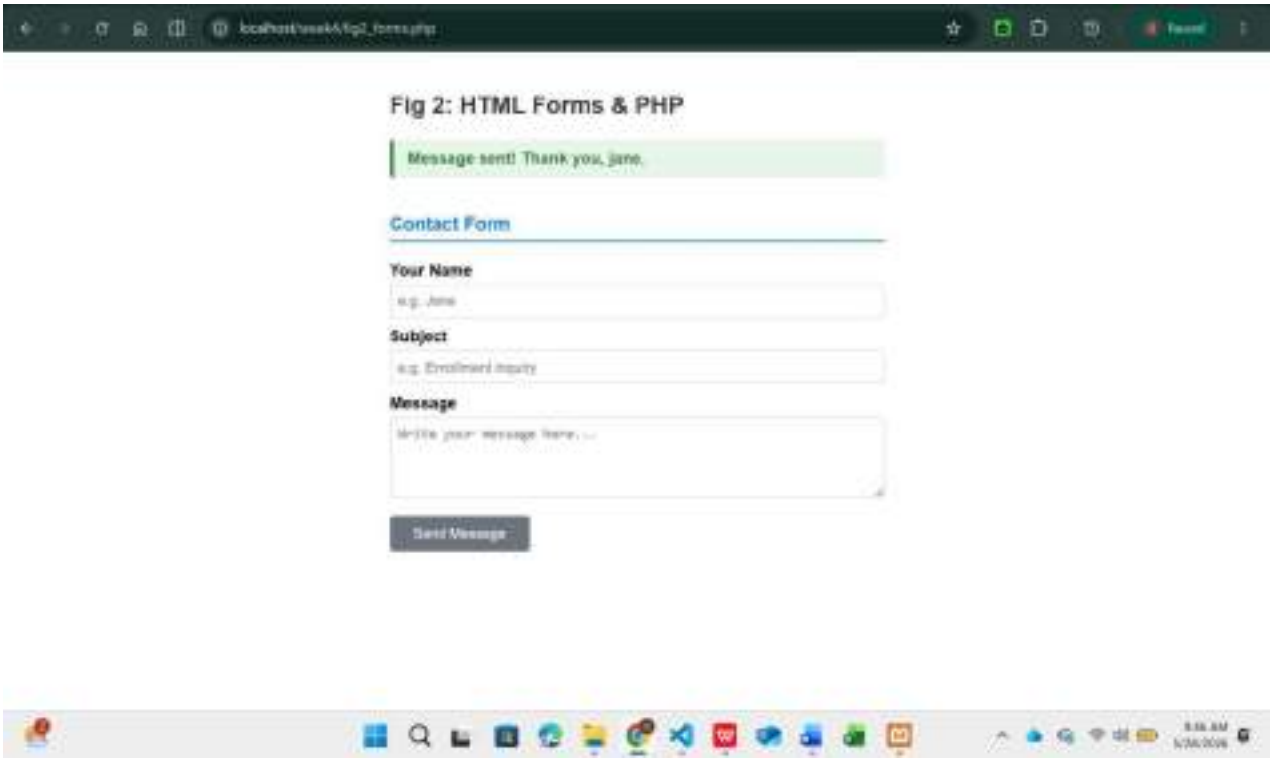

Fig 4: Shows the HTML Forms and PHP Contact Form displaying a successful message submission confirmation.

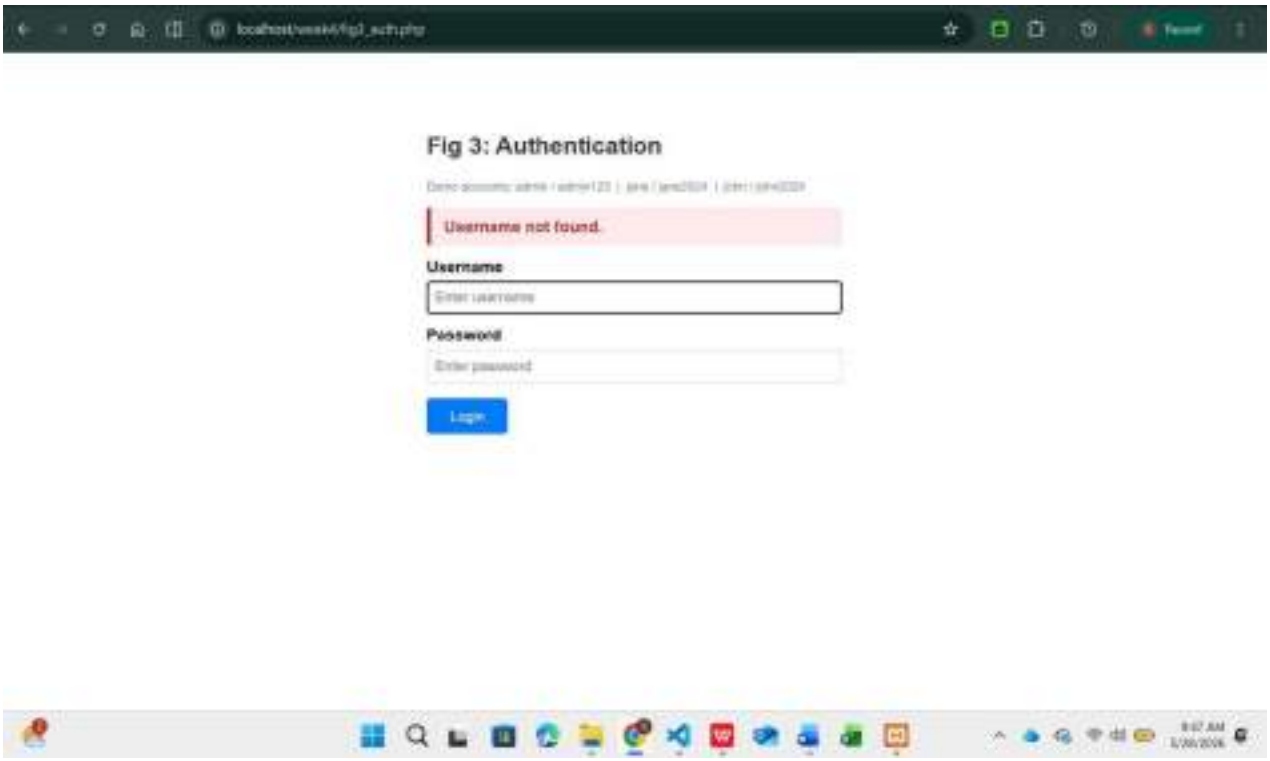

Fig 5: Illustrates the PHP Authentication page displaying an error message when an unrecognized username is entered.

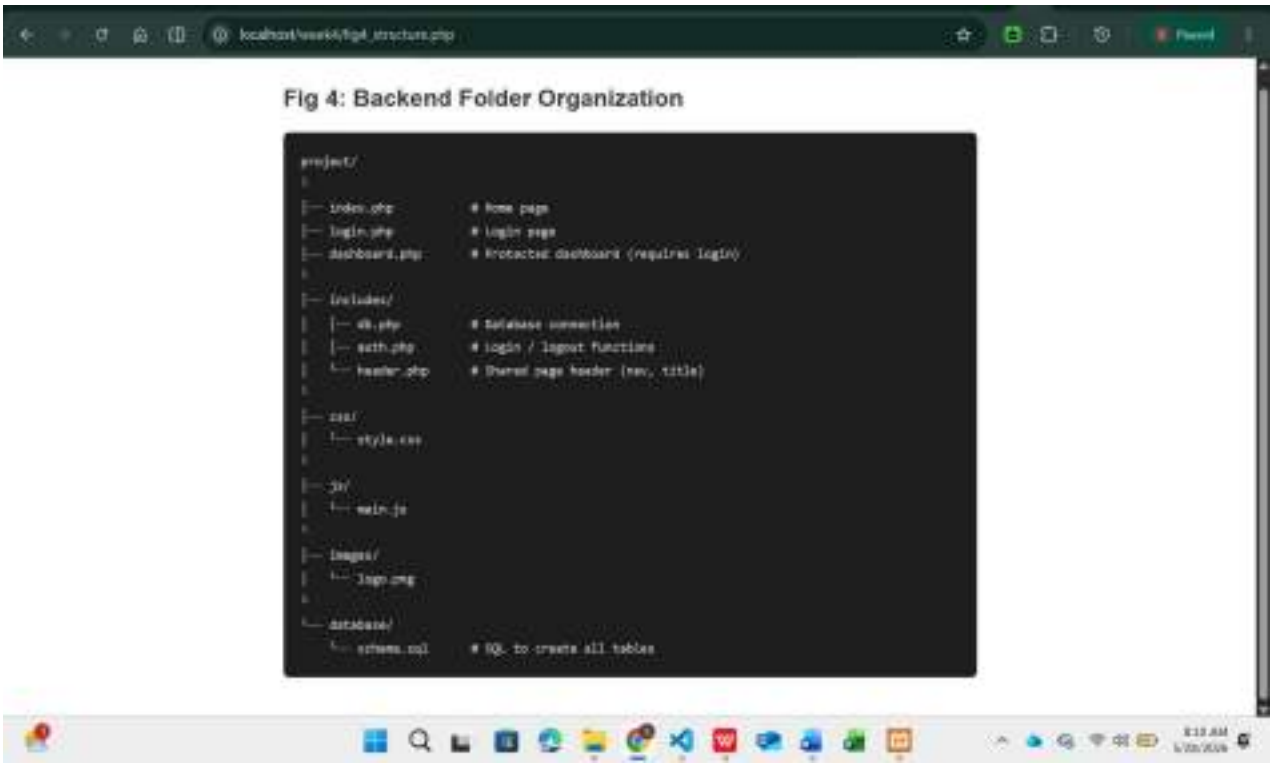

Fig 6 Shows the backend folder organization structure, outlining the project directory and its key files and subdirectories.

My Reflection

I focused on building interactive web forms and PHP-based authentication. I developed registration, login, and contact forms with server-side validation, session management, and role-based access for admin and regular users. Organizing the backend into clear folders improved maintainability, while handling form errors smoothly was the main challenge.

WEEK 5: DATABASE SYSTEMS

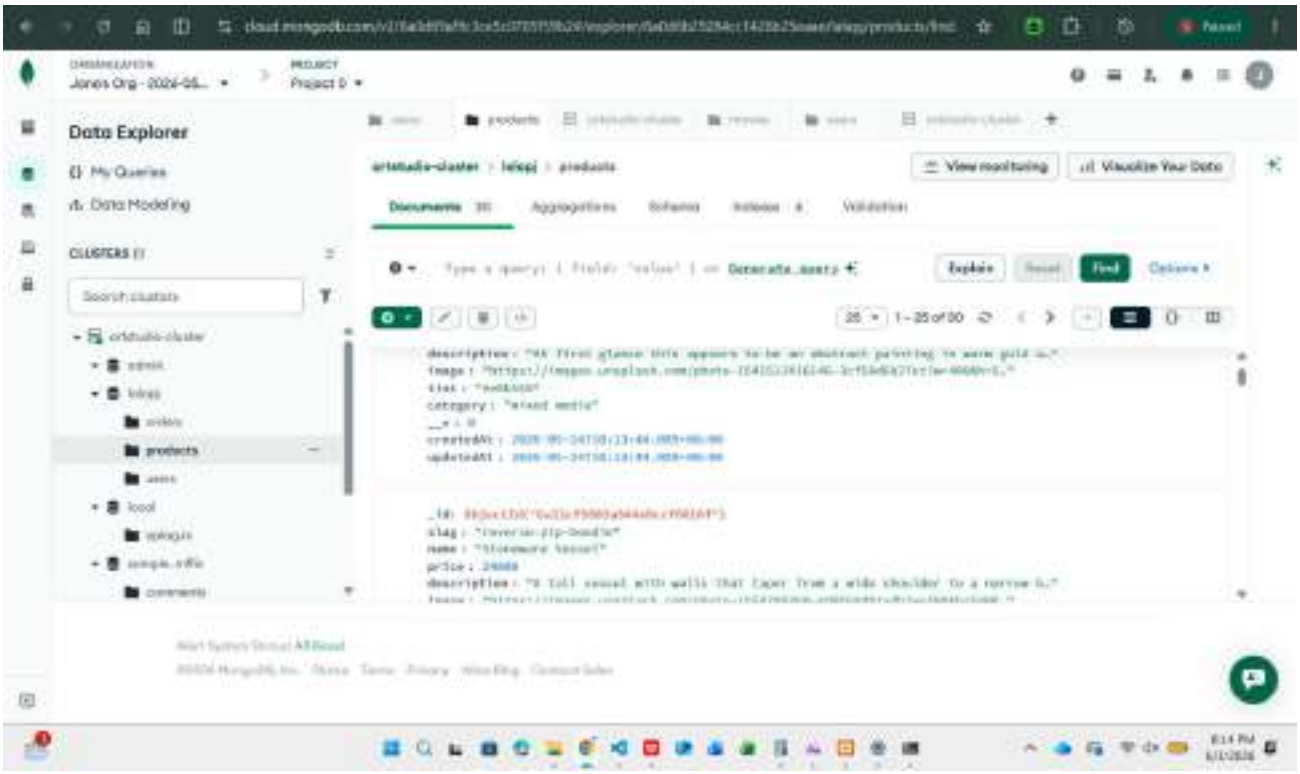

Fig 1: MongoDB Atlas Data Explorer showing the art studio database

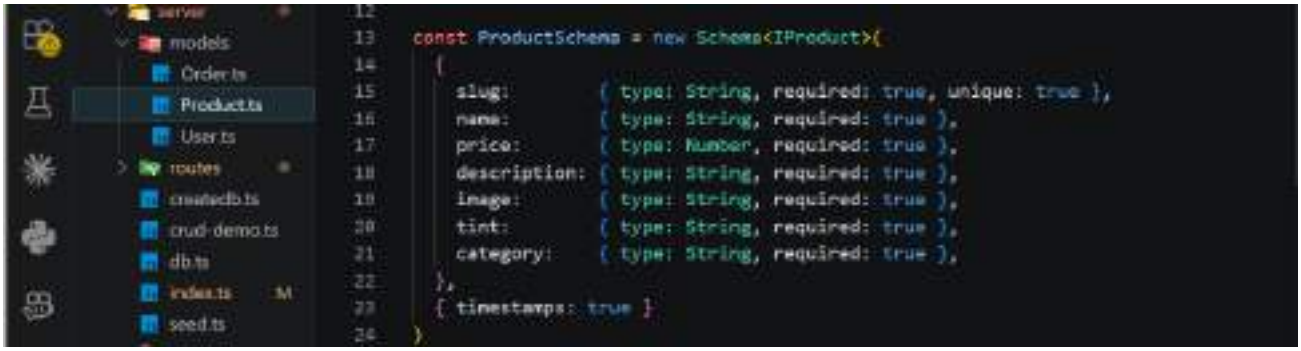

*Fig 2: Mongoose ProductSchema defining the structure of the users collection***.**

```
CREATE — INSERT INTO products
SQL: INSERT INTO products(slug, name, price, category) VALUES('demo-art', 'Demo Art', 5000, 'painting');
✓ Product created:
  _id      : 6a206b2be003346137db557b
  slug     : demo-art
  name     : Demo Art
  price    : KSh 5,000
  category: painting
  createdAt: Wed Jun 03 2026 20:58:03 GMT+0300 (East Africa Time)
```

Fig 3a: Terminal confirms a new product "Demo Art" was successfully created.

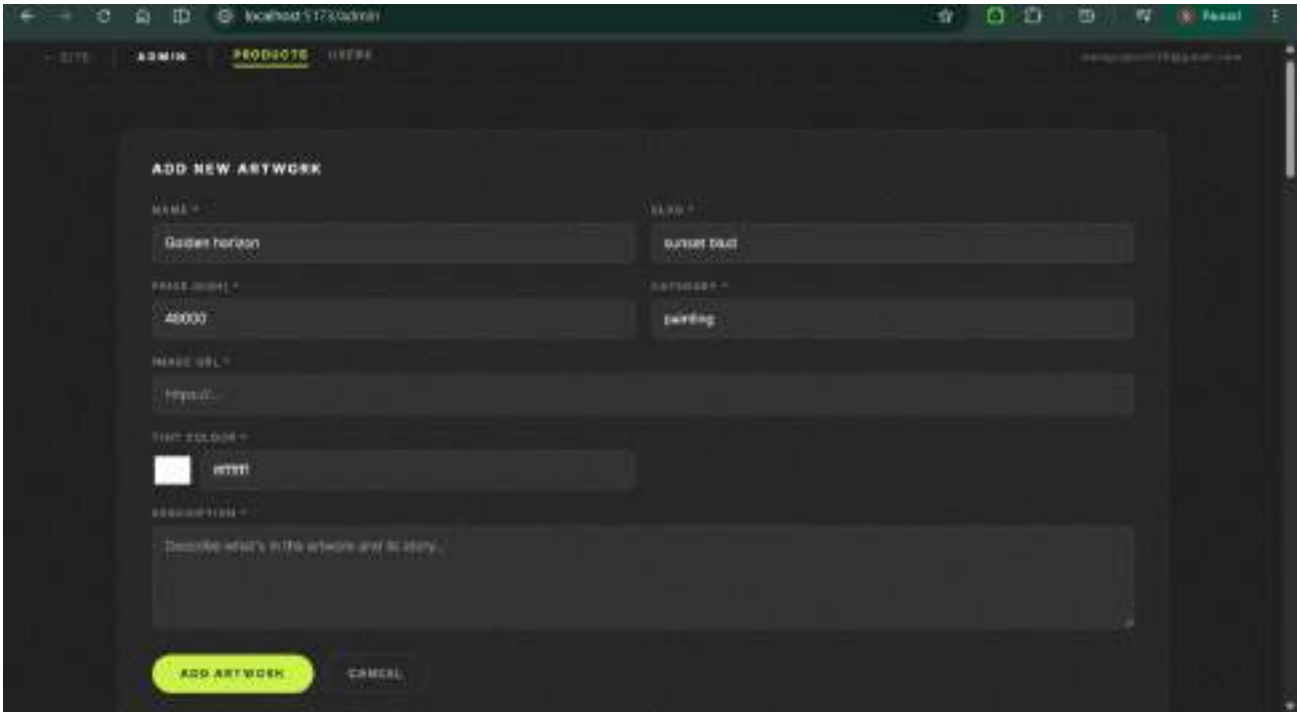

Fig 3b: Admin form used to add new artwork to the database.

```
PS C:\Users\murin\code\ARTSTUDIO> cd C:\Users\murin\code\ARTSTUDIO\PROJECT
>> npm run crud-demo
>>
READ — SELECT * FROM products
   SQL: SELECT slug, name, price, category FROM products;
   ✓ Found 31 products:
     [1] reverse-joggers  —  Fractured Light  —  KSh 89,000  (mixed media)
     [2] reverse-zip-hoodie  —  Stoneware Vessel  —  KSh 24,000  (ceramics)
     [3] slouch-socks  —  Neon Grid  —  KSh 9,500  (digital)
     [4] rest-slippers-fresh  —  Verdant Study  —  KSh 54,000  (painting)
     [5] sizzler-case-pink  —  Rose Noise  —  KSh 16,000  (print)
     [6] beach-slides-blue  —  Ocean Series II  —  KSh 49,000  (painting)
     [7] bucket-hat  —  Golden Hour  —  KSh 40,000  (photography)
     [8] reverse-hoodie-iris  —  Bloom Series I  —  KSh 18,000  (print)
     [9] rest-slides-sand  —  Shadow Play  —  KSh 38,000  (photography)
     [10] rest-slippers-pie  —  Amber Resin Panel  —  KSh 11,000  (mixed media)
     [11] bump-case-blue  —  Storm Study  —  KSh 74,000  (painting)
     [12] pleated-denim-indigo  —  Deep Indigo  —  KSh 75,000  (painting)
     [13] beanie-iris  —  Midnight Abstract  —  KSh 62,000  (painting)
     [14] reverse-hoodie-lime  —  Urban Silence  —  KSh 32,000  (photography)
     [15] reverse-sweats  —  Forest Echo  —  KSh 56,000  (painting)
     [16] rest-slides-dream  —  Still Life No. 7  —  KSh 42,000  (painting)
     [17] rest-slides-bloop  —  Coastal Morning  —  KSh 15,000  (print)
     [18] sizzler-case-jet  —  Monochrome No. 4  —  KSh 44,000  (photography)
     [19] denim-shorts  —  Indigo Sky  —  KSh 36,000  (photography)
     [20] pleated-denim-light  —  Morning Haze  —  KSh 13,000  (print)
     [21] beach-slides-cream  —  Pale Light  —  KSh 29,000  (photography)
```

Fig 4a: Terminal retrieves all 31 products from the MongoDB products collection.

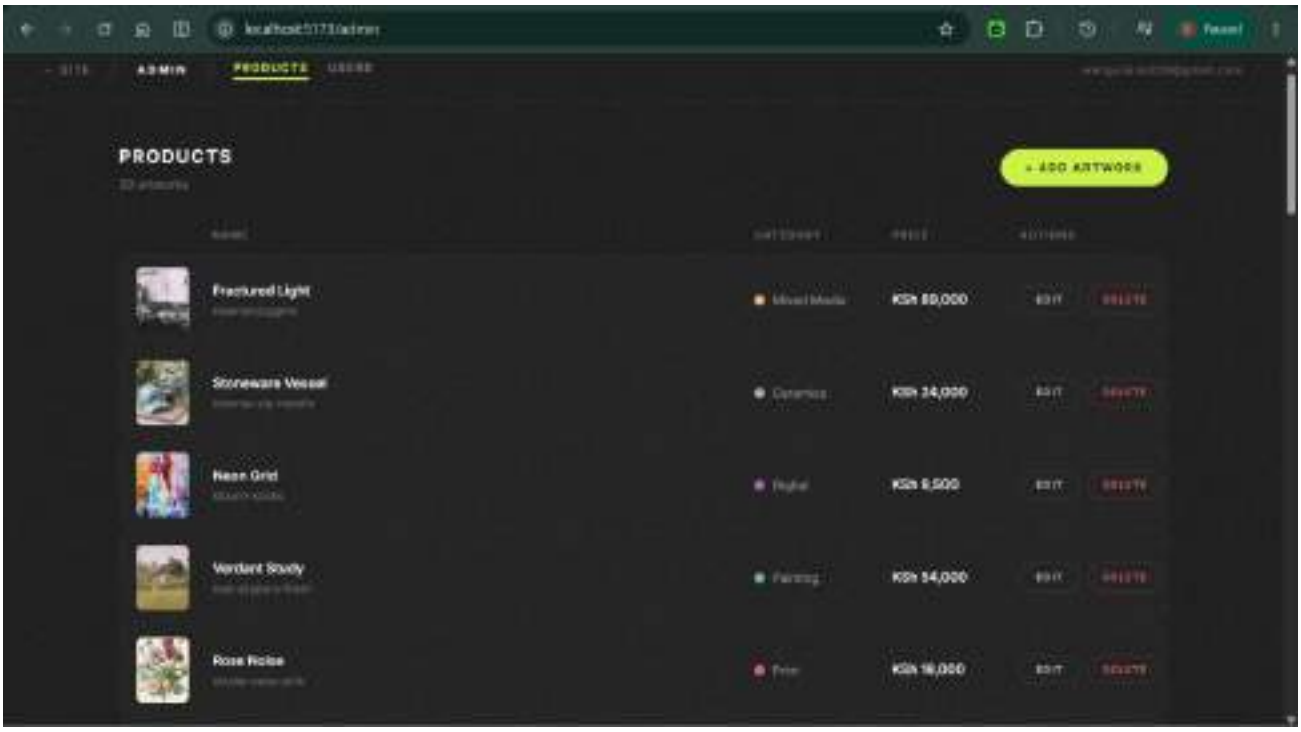

Fig 4b: Admin dashboard displays all 30 artworks fetched from MongoDB.

```
UPDATE — UPDATE products SET price
   SQL: UPDATE products SET price = 7500 WHERE slug = 'demo-art';
   ✓ Product updated:
     slug     : demo-art
     old price: KSh 5,000
     new price: KSh 7,500
     updatedAt: Wed Jun 03 2026 21:08:07 GMT+0300 (East Africa Time)
```

Fig 5a: Terminal confirms a product document was successfully updated in MongoDB.

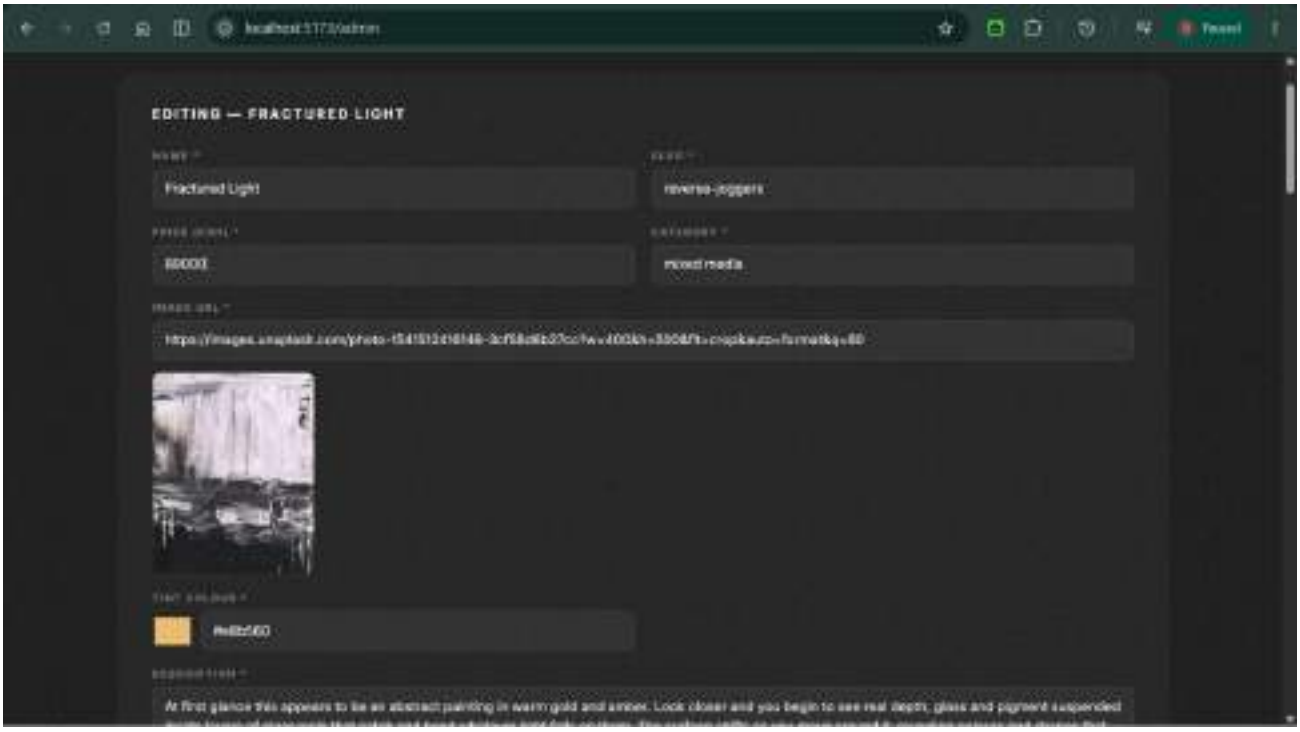

Fig 5b: Admin panel shows EDIT button for updating any product record.

```
DELETE — DELETE FROM products
   SQL: DELETE FROM products WHERE slug = 'demo-art';
   ✓ Product deleted. Still exists? NO
```

Fig 6a: Terminal confirms a product document was successfully deleted from MongoDB.

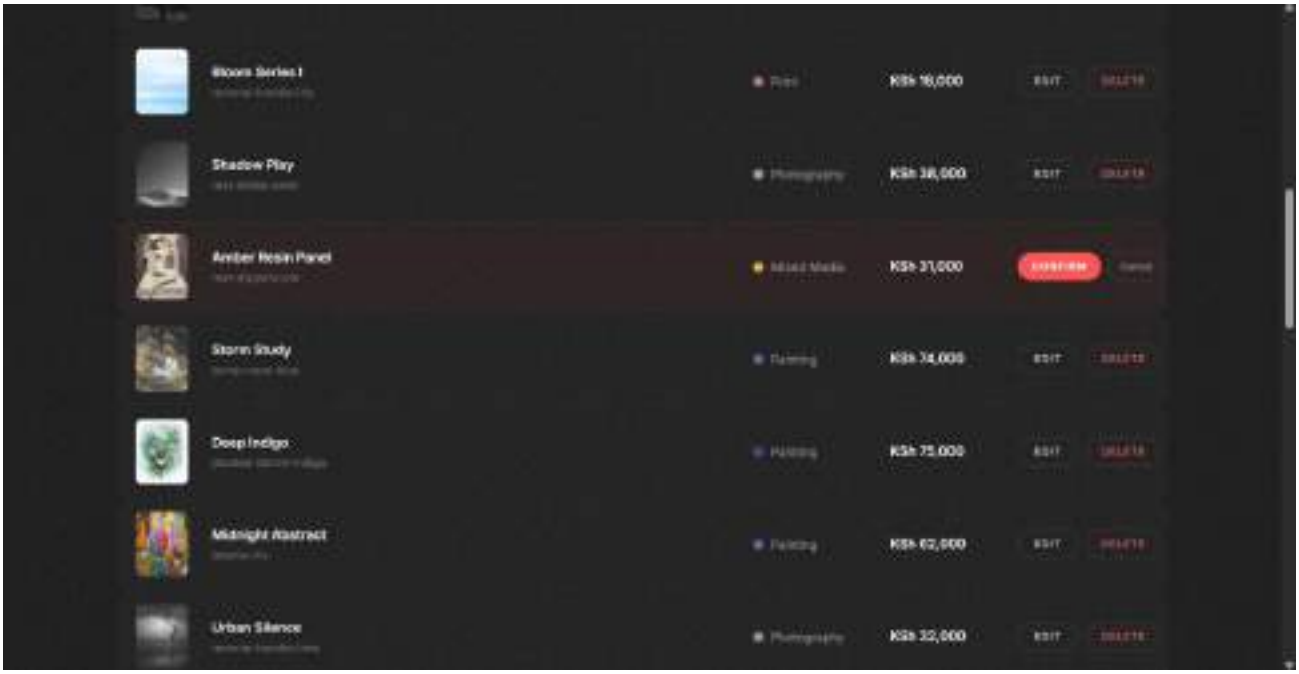

Fig 6b: Admin panel shows DELETE button for removing any product record.

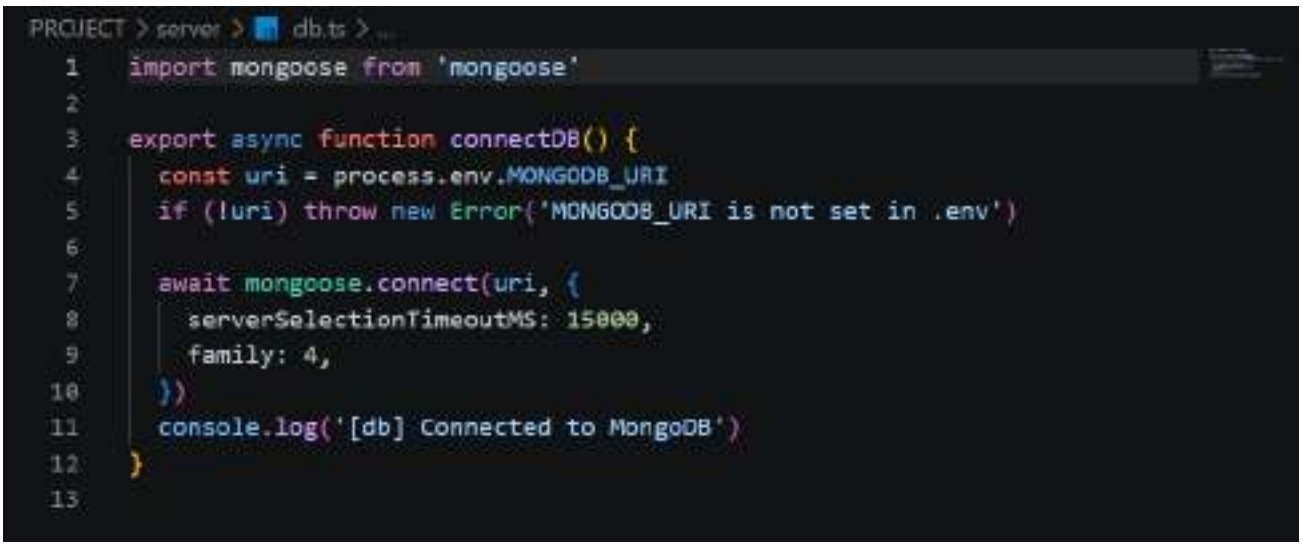

Fig 7a : db.ts defines connectDB () to securely connect to MongoDB using Mongoose.

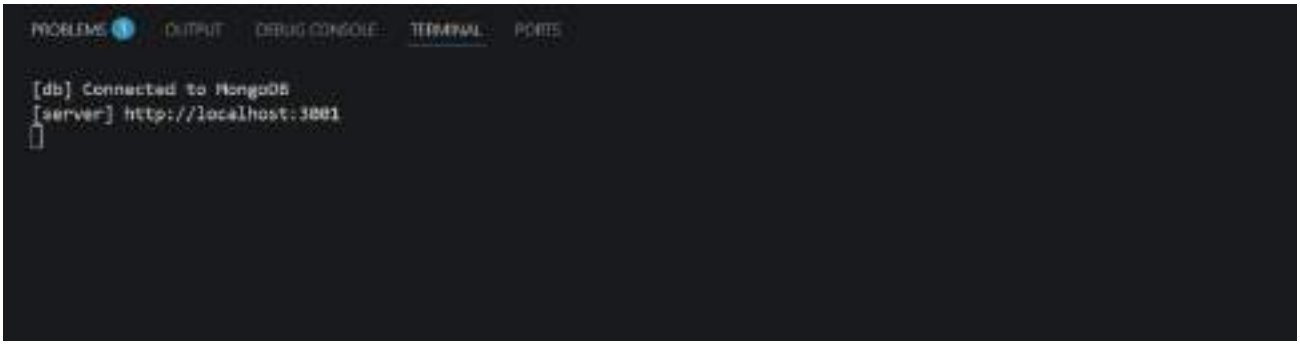

Fig 7b: Terminal confirms successful MongoDB connection with server running on localhost:3001

.

```
5
6   router.get('/products', async (_, res) => {
7     try {
8       const products = await Product.find().lean()
9       res.json(products)
10    } catch (err) {
11      res.status(500).json({ error: 'Failed to fetch products' })
12    }
13  })
14
15  router.get('/products/:slug', async (req, res) => {
16    try {
17      const product = await Product.findOne({ slug: req.params.slug }).lean(
18      if (!product) return res.status(404).json({ error: 'Not found' })
19      res.json(product)
20    } catch (err) {
21      res.status(500).json({ error: 'Failed to fetch product' })
22    }
23  })
```

Fig 8: Product. Find() fetches all products from the MongoDB products collection.

My Reflection

I gained a deeper understanding of databases and their role in data-driven web applications. In my Art Studio project, I used MongoDB instead of MySQL, created collections, implemented CRUD operations, and connected the database securely using Mongoose and environment variables. This strengthened my ability to manage and retrieve real-world data effectively.

WEEK 6: DATABASE INTEGRATION AND CRUD OPERATIONS

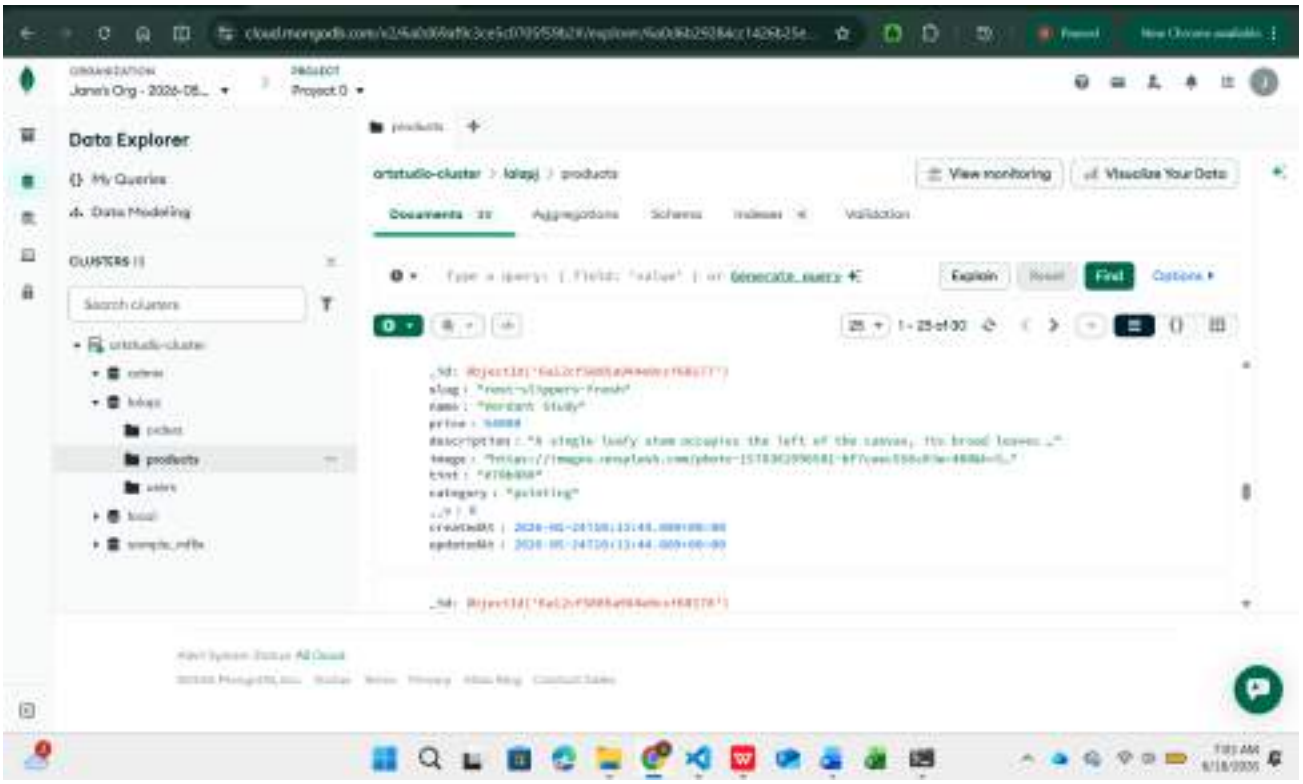

Fig 1a: MongoDB atlas showing the products collection with 30 documents in the database

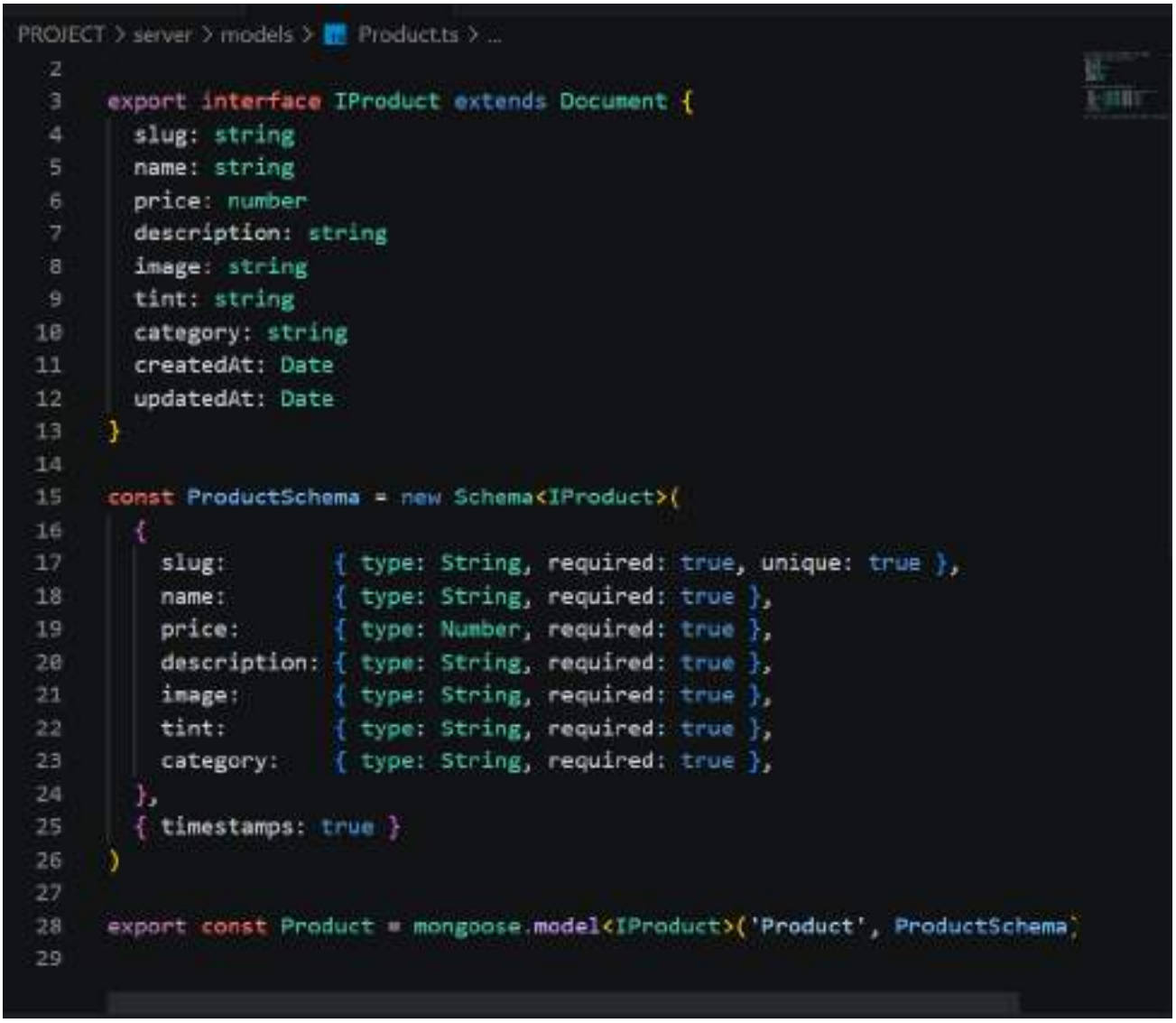

Fig 1b: Mongoose database schema in the vs code with typescript interface

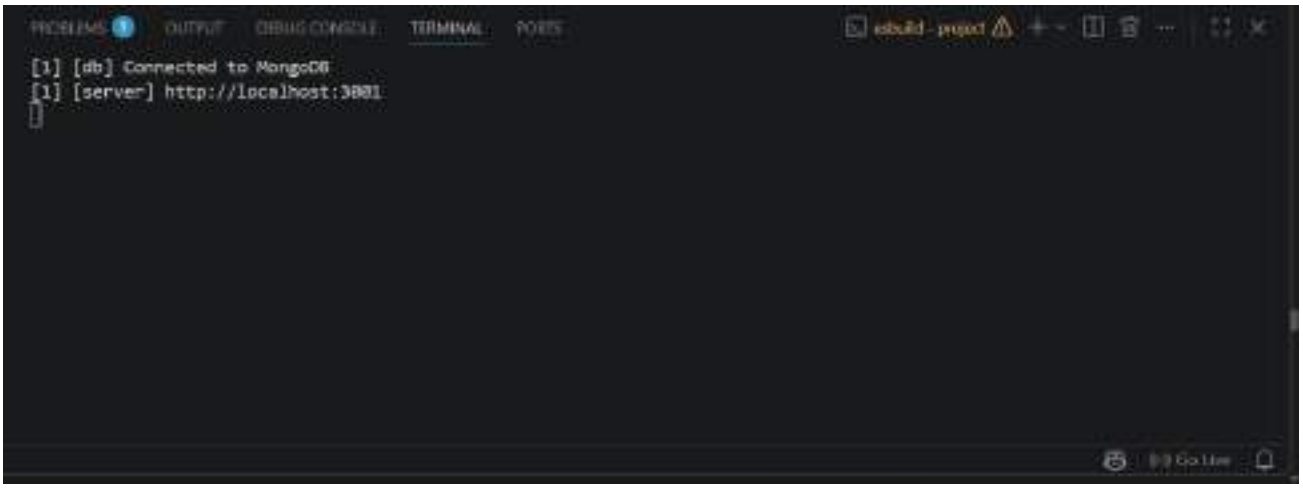

Fig 2: Terminal output showing successful server startup and MongoDB atlas database connection

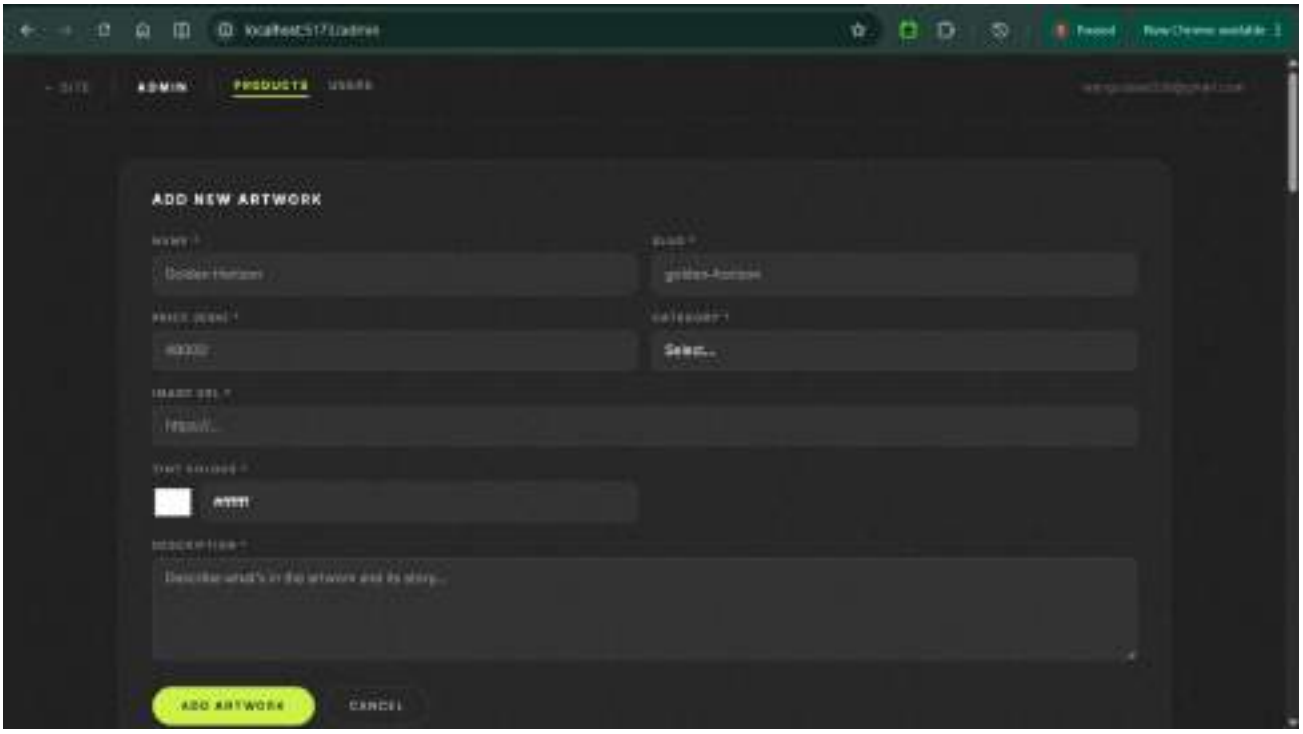

Fig 3a: Admin panel form used to add new artwork record fields for name, slug, price,category and description

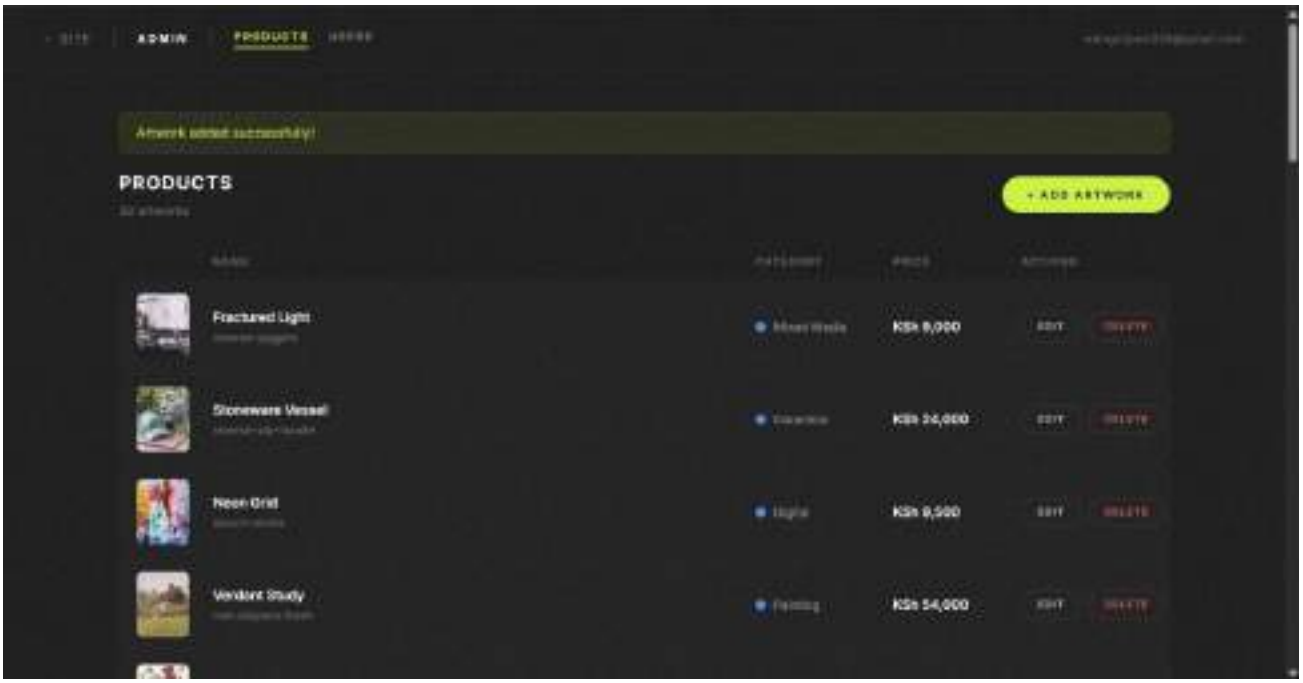

Fig 3b: Admin product dashboard displaying a successful message after adding new artwork

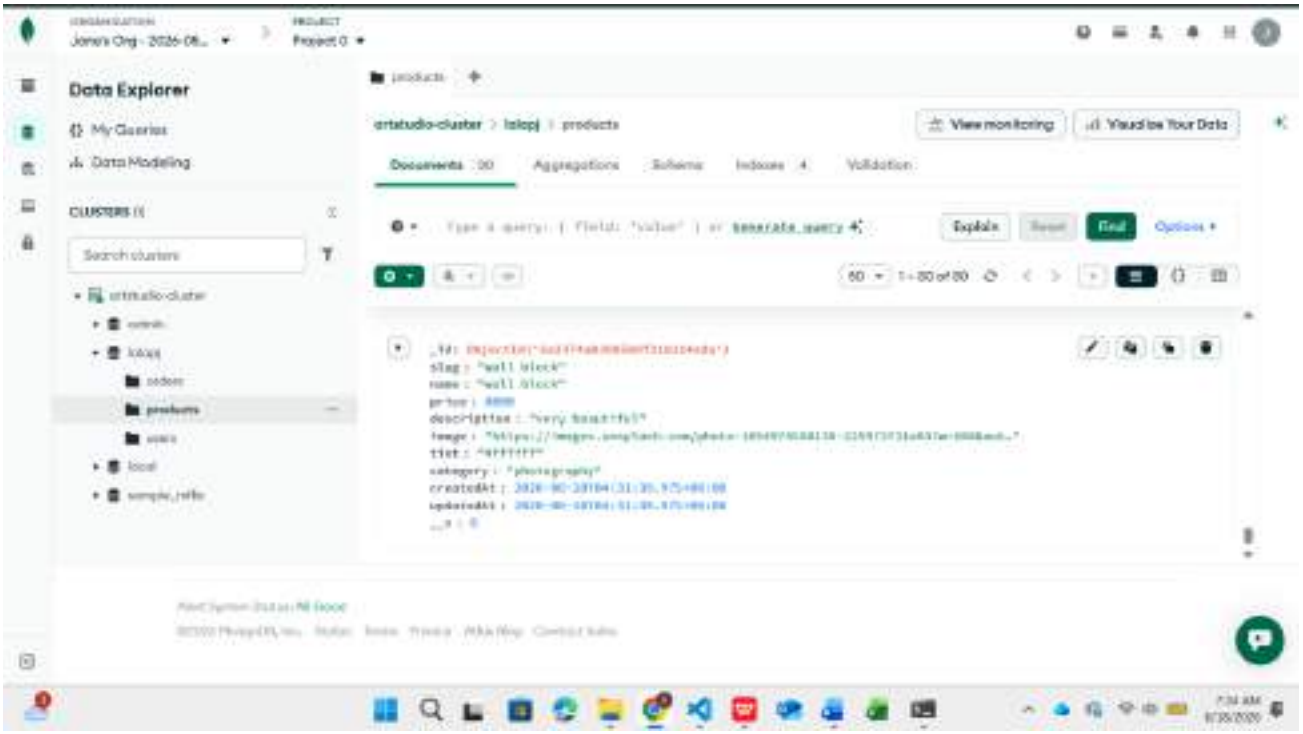

Fig 3c: MongoDB atlas confirming the newly inserted artwork in the product collection

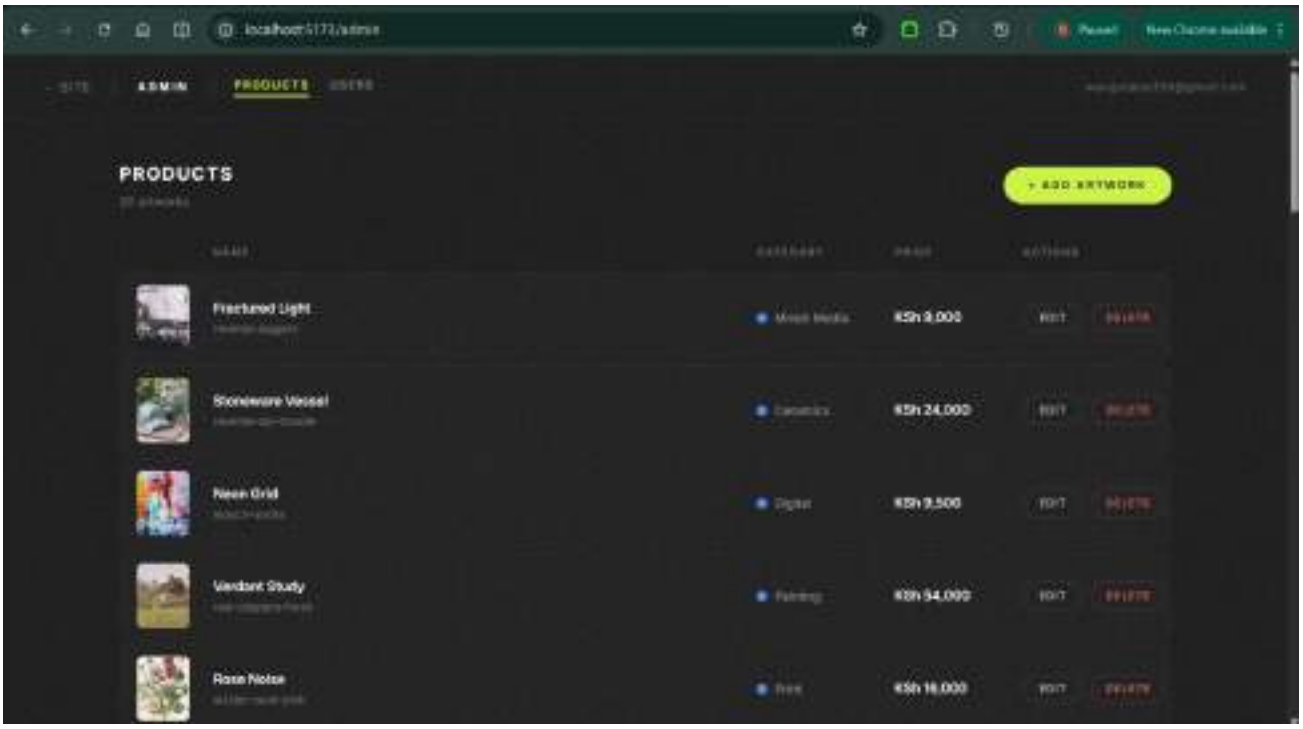

Fig 4: Admin dashboard showing and displaying all the artwork

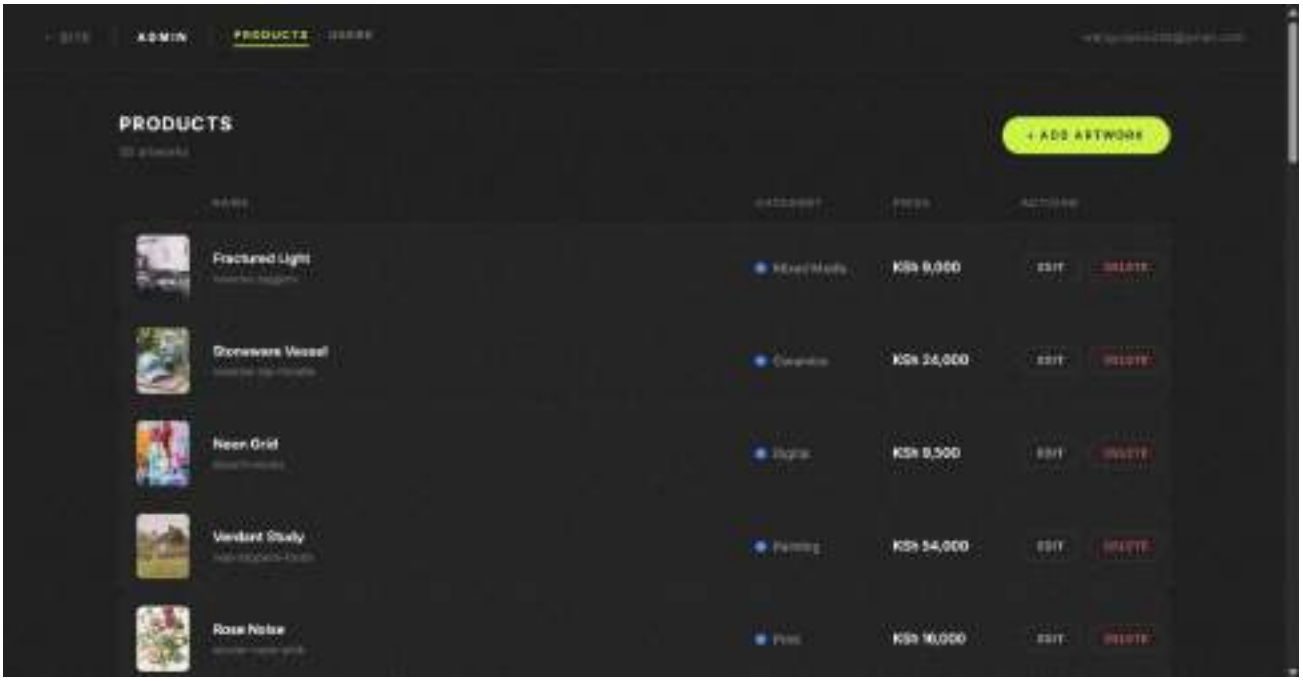

Fig 5a: Products list showing "Fractured Light" at KSh 9,000 before the update operation is performed.

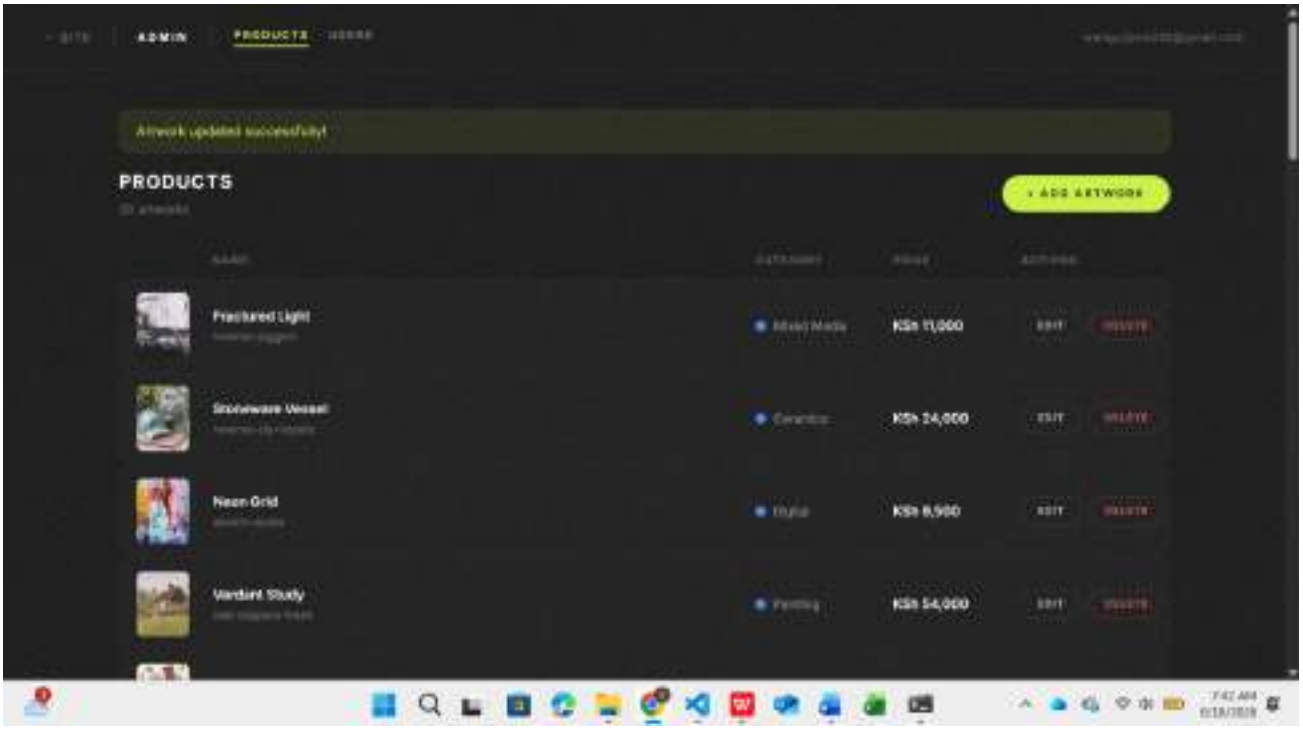

Fig 5b: product list showing "Fractured light " at ksh 11000 after editing

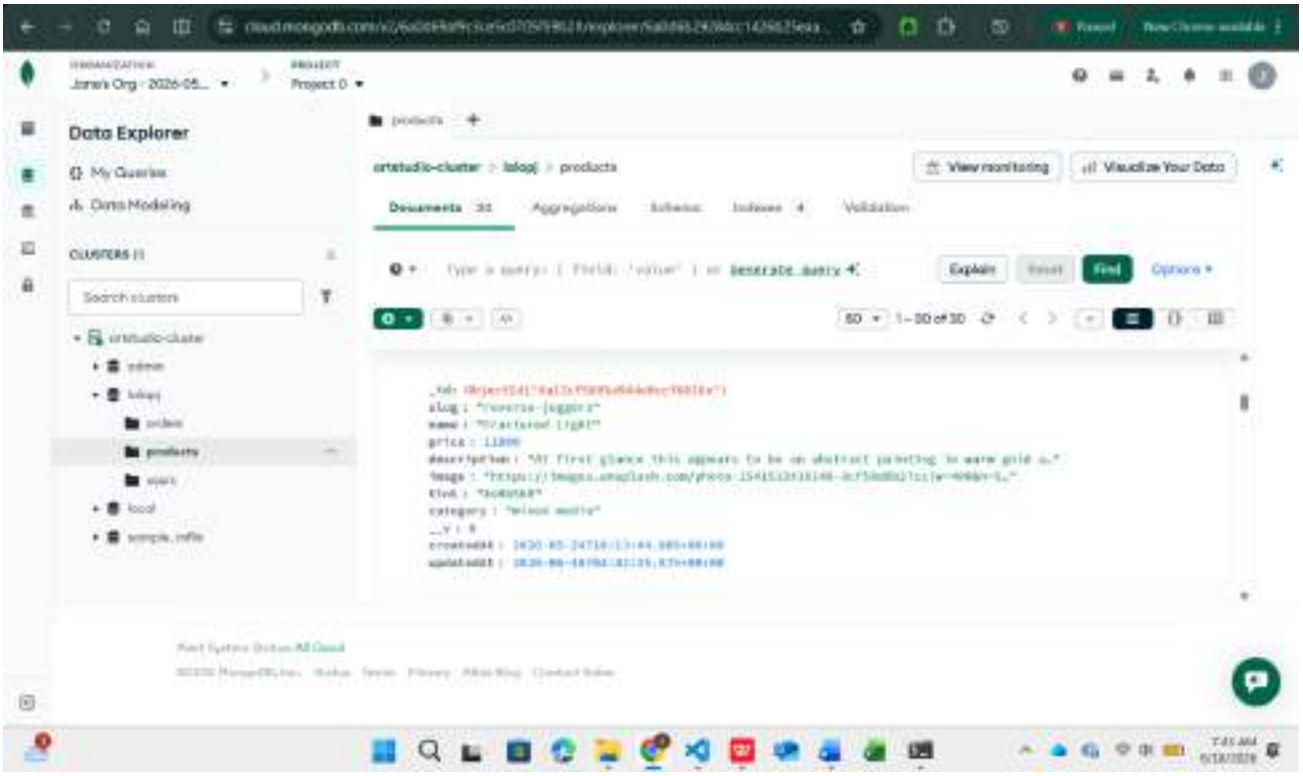

Fig 5c: MongoDB Atlas showing the change made in the browser has also reflected in the db

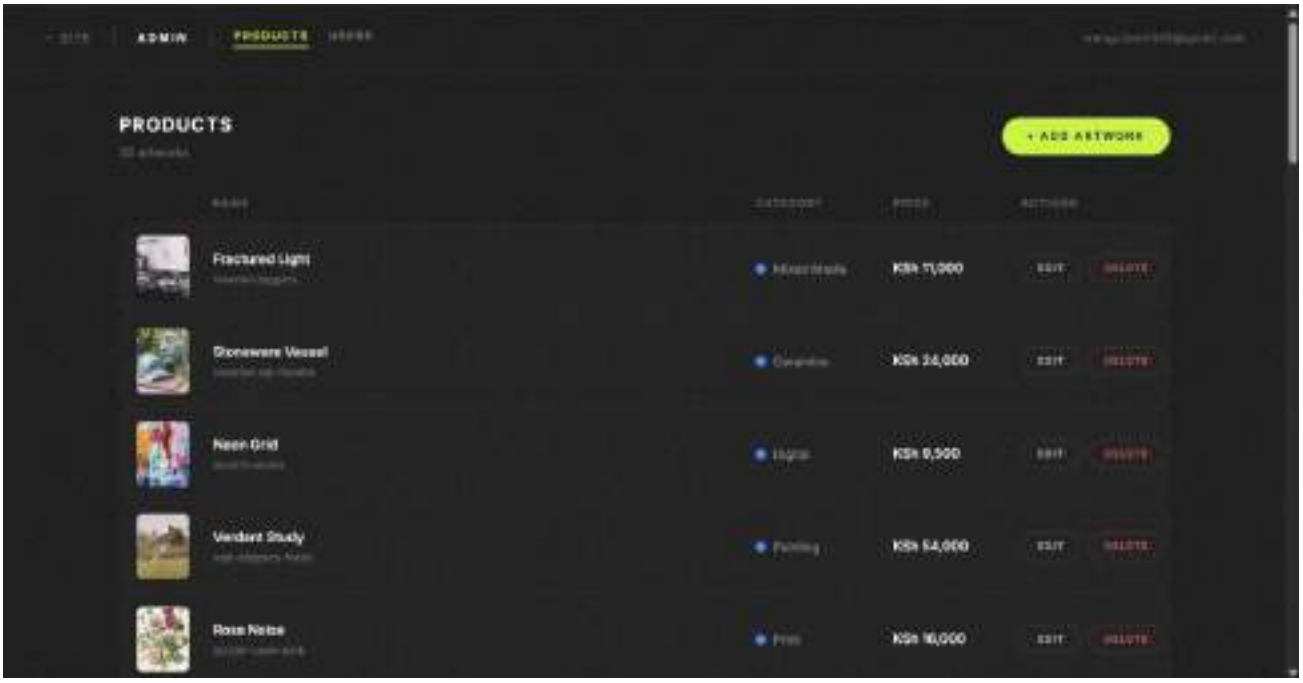

Fig 6a: Products list showing 30 artworks before the DELETE operation, with the "wall block" test product to be removed by scrolling down.

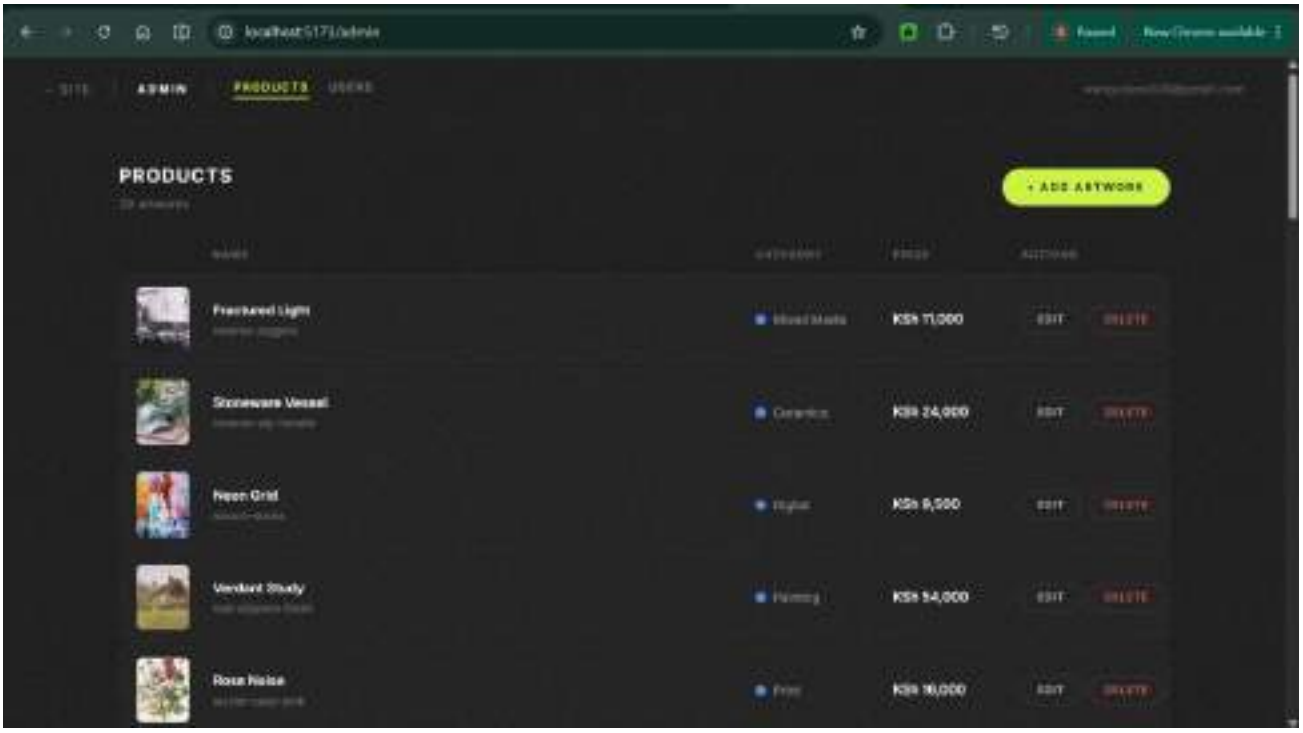

Fig 6b: Products list now showing 29 artworks after the DELETE operation successfully removed the "wall block" document from the MongoDB database.

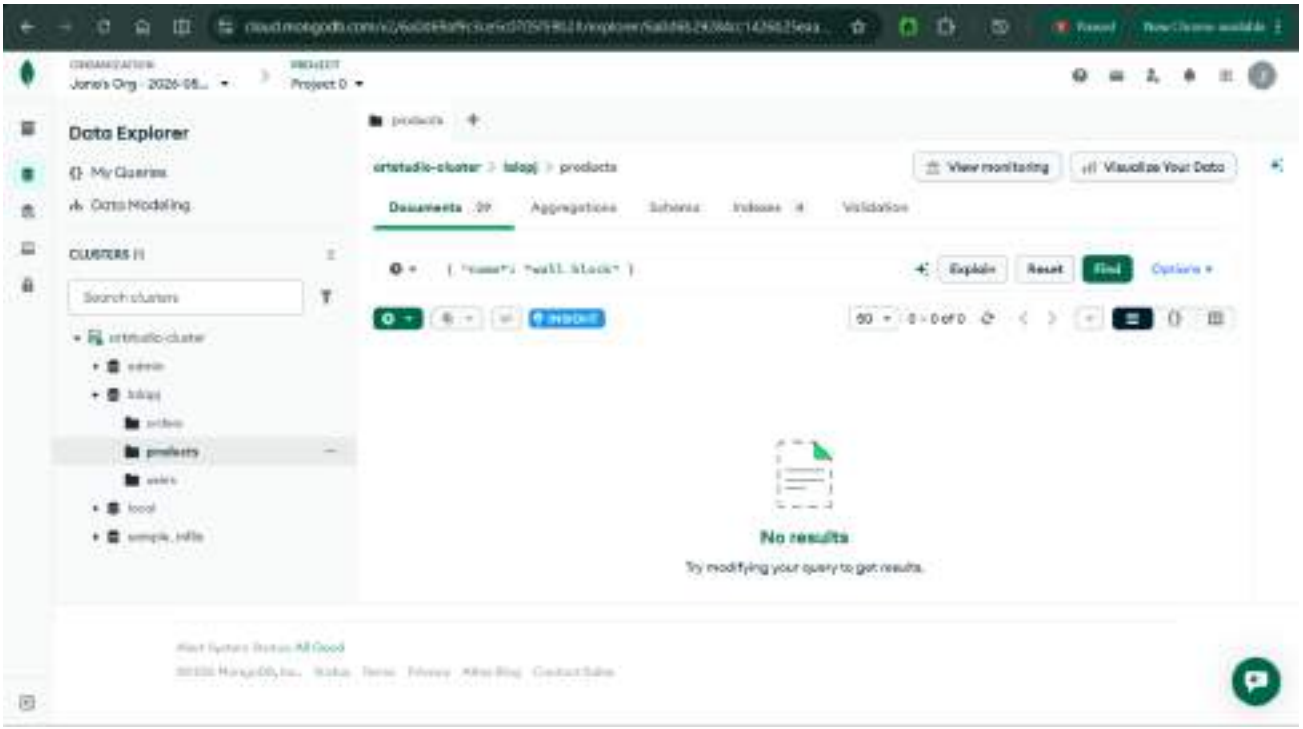

Fig 6c: MongoDB Atlas search query {"name": "wall block"} returning 0 results and 29 documents remaining, confirming the document was permanently deleted from the database.

My Reflection

This practical introduced me to the MEVN stack - MongoDB, Express, Vue.js. I successfully implemented all four CRUD operations, creating new network records through an admin form, reading and displaying them dynamically on the front end, updating existing records with real time feedback, and deleting records permanently from the database.

WEEK 7: USER AUTHENTICATION AND SESSION MANAGEMENT

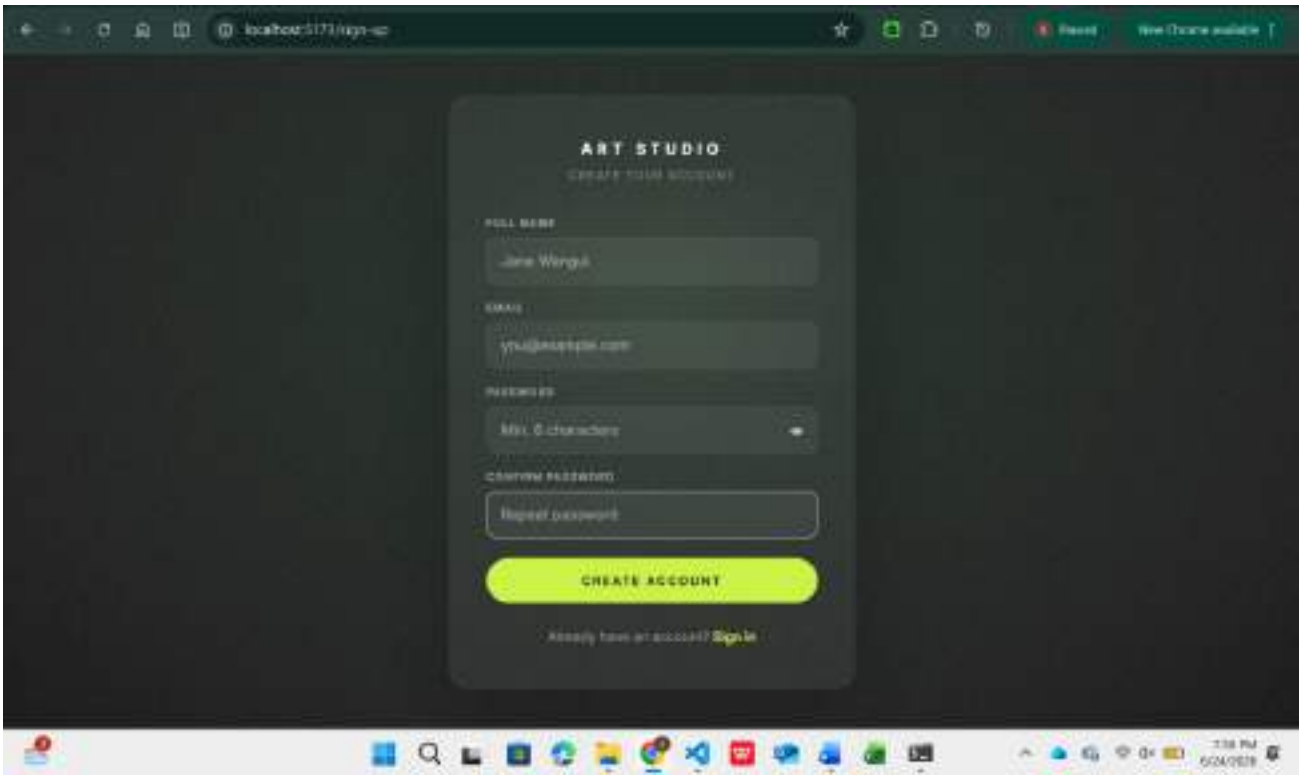

Fig 1: Art Studio sign-in page displaying the user login form

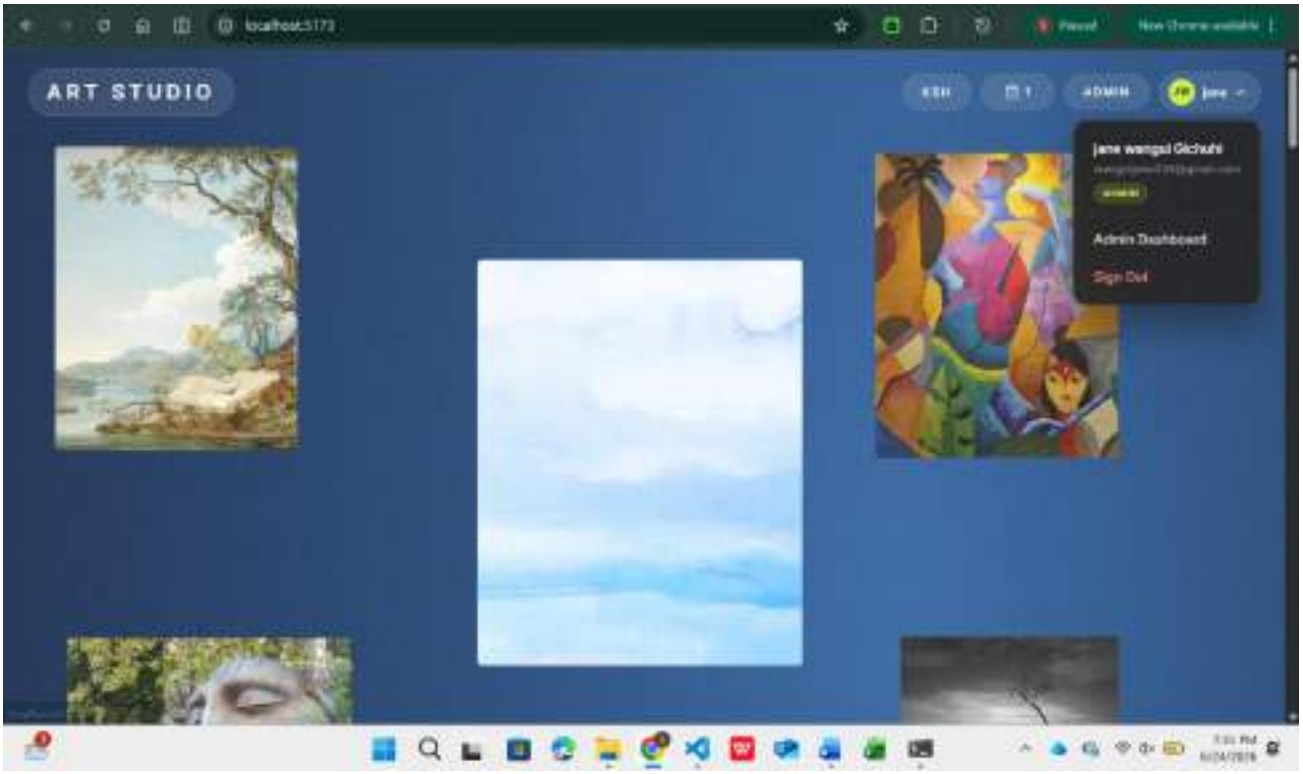

Fig 2: Art Studio homepage displaying the authenticated user session with the logged-in user's profile dropdown showing name, email, admin role, and sign-out option.

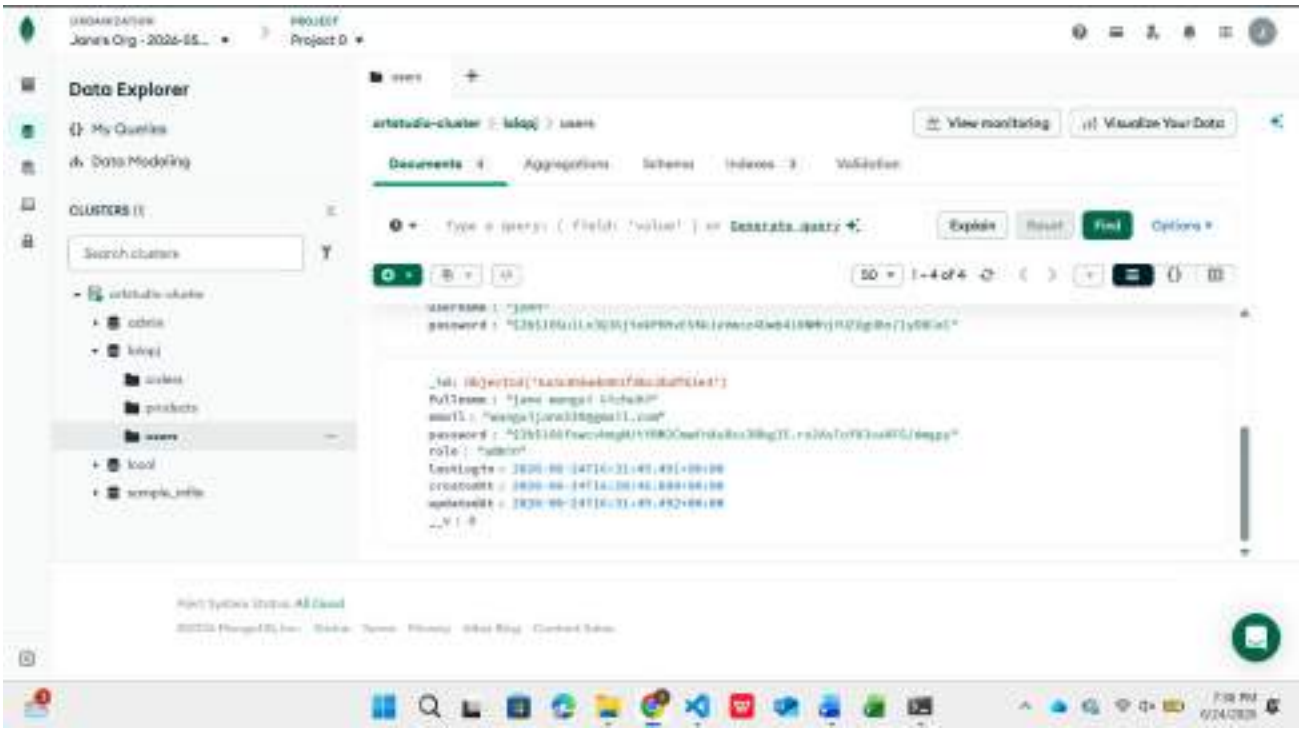

Fig 3: MongoDB Atlas Data Explorer showing the users collection with bcrypt-hashed passwords stored in the database.

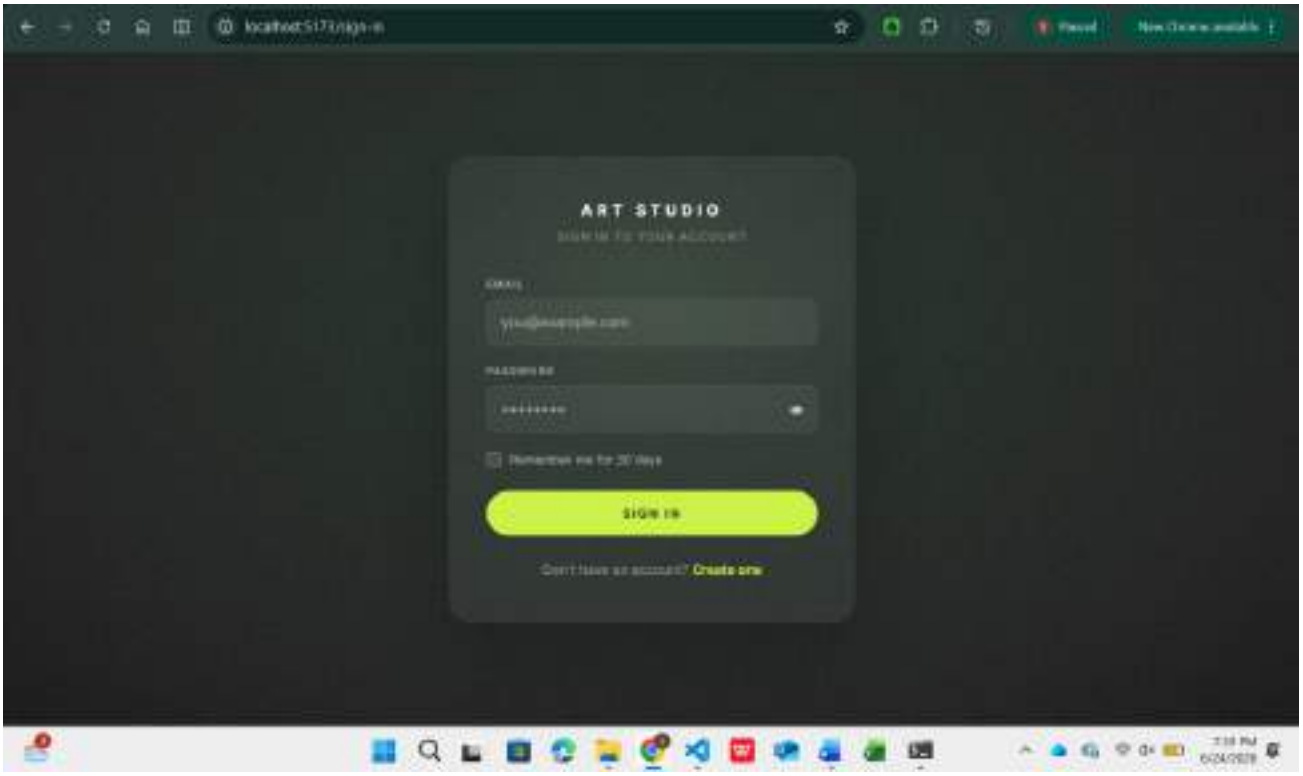

Fig 4: Art Studio sign-in page redirected to after logout, confirming successful session termination.

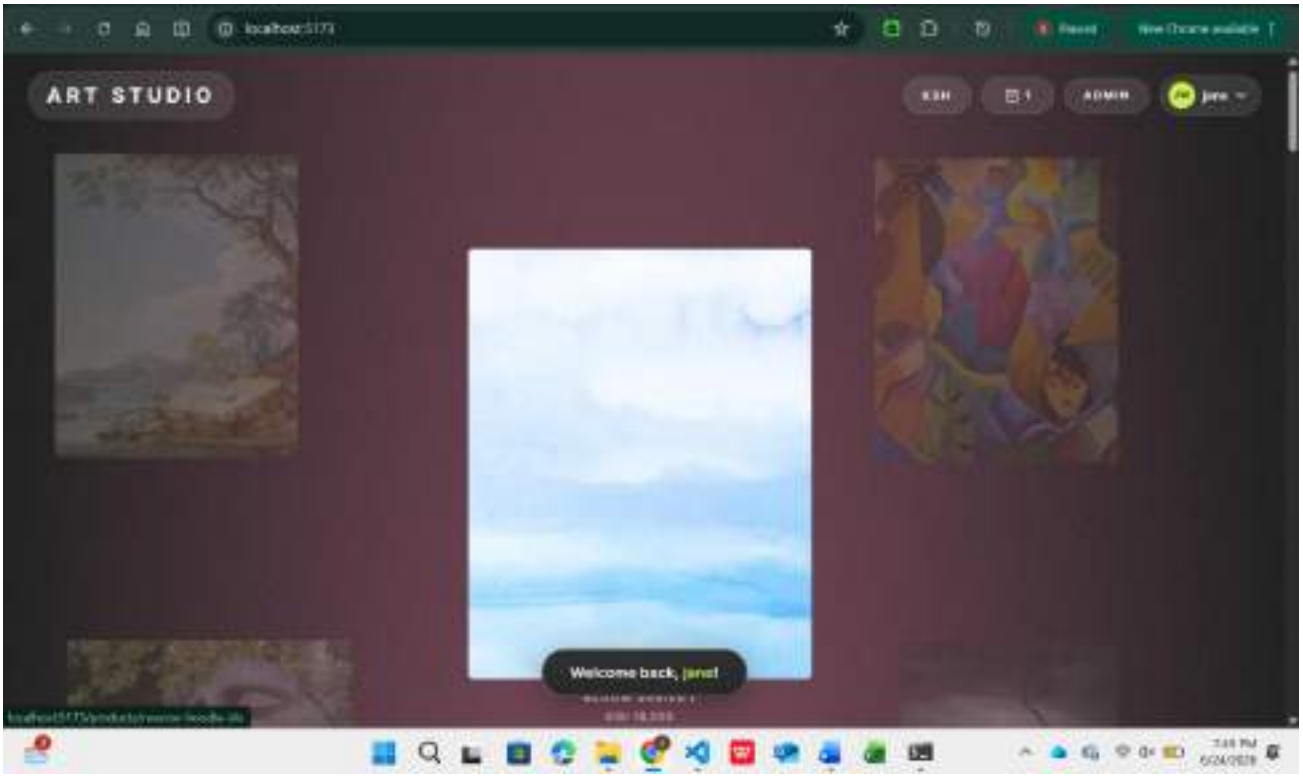

Fig 5: Dashboard showing the "Welcome back, jane!"notification confirming a successful user login session.

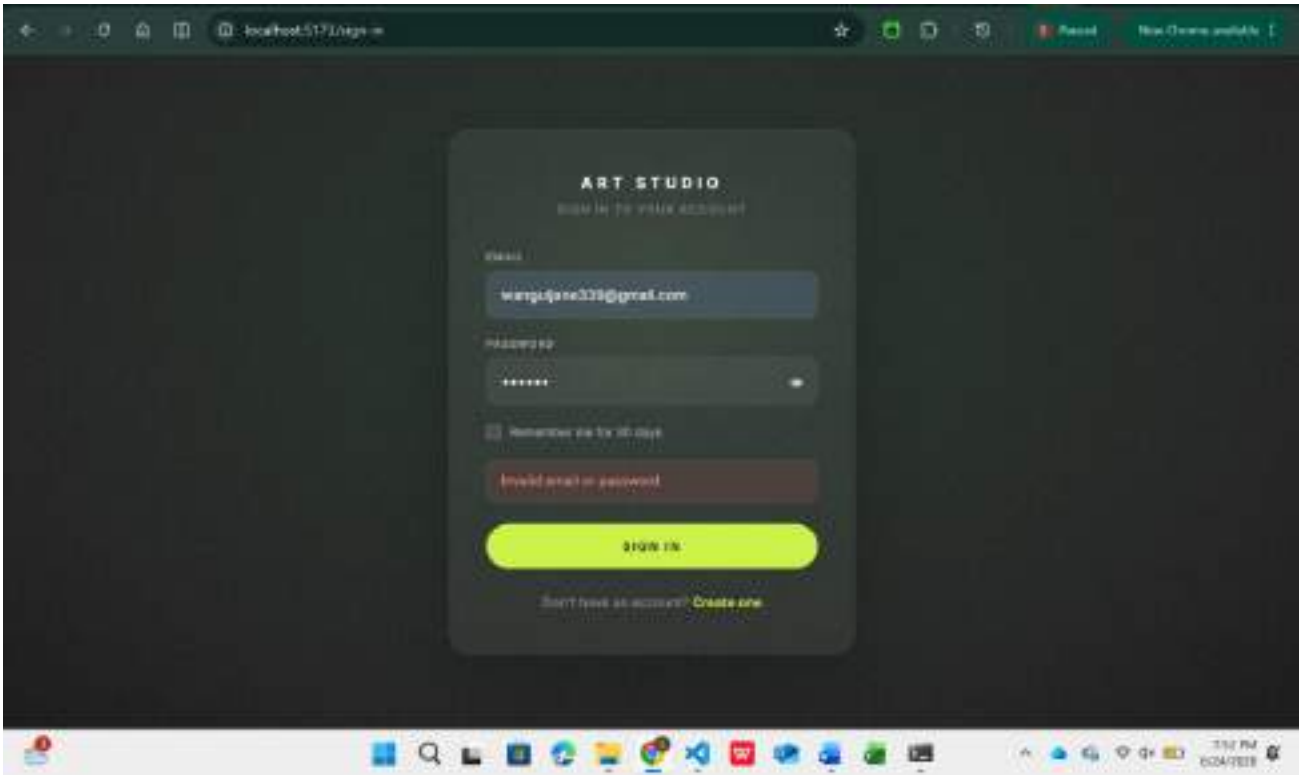

Fig 7: Art Studio sign-in page displaying an "Invalid email or password" error message after submitting incorrect credentials.

My Reflection

Through implementing user authentication and session management, I learned how to protect web applications from unauthorized access. I created registration and login features, used password hashing to store passwords securely, managed sessions for active users, and controlled access based on roles. This improved my understanding of secure backend development practices.

WEEK 8 RESPONSIVE WEB DESIGN AND MOBILE-FIRST DEVELOPMENT

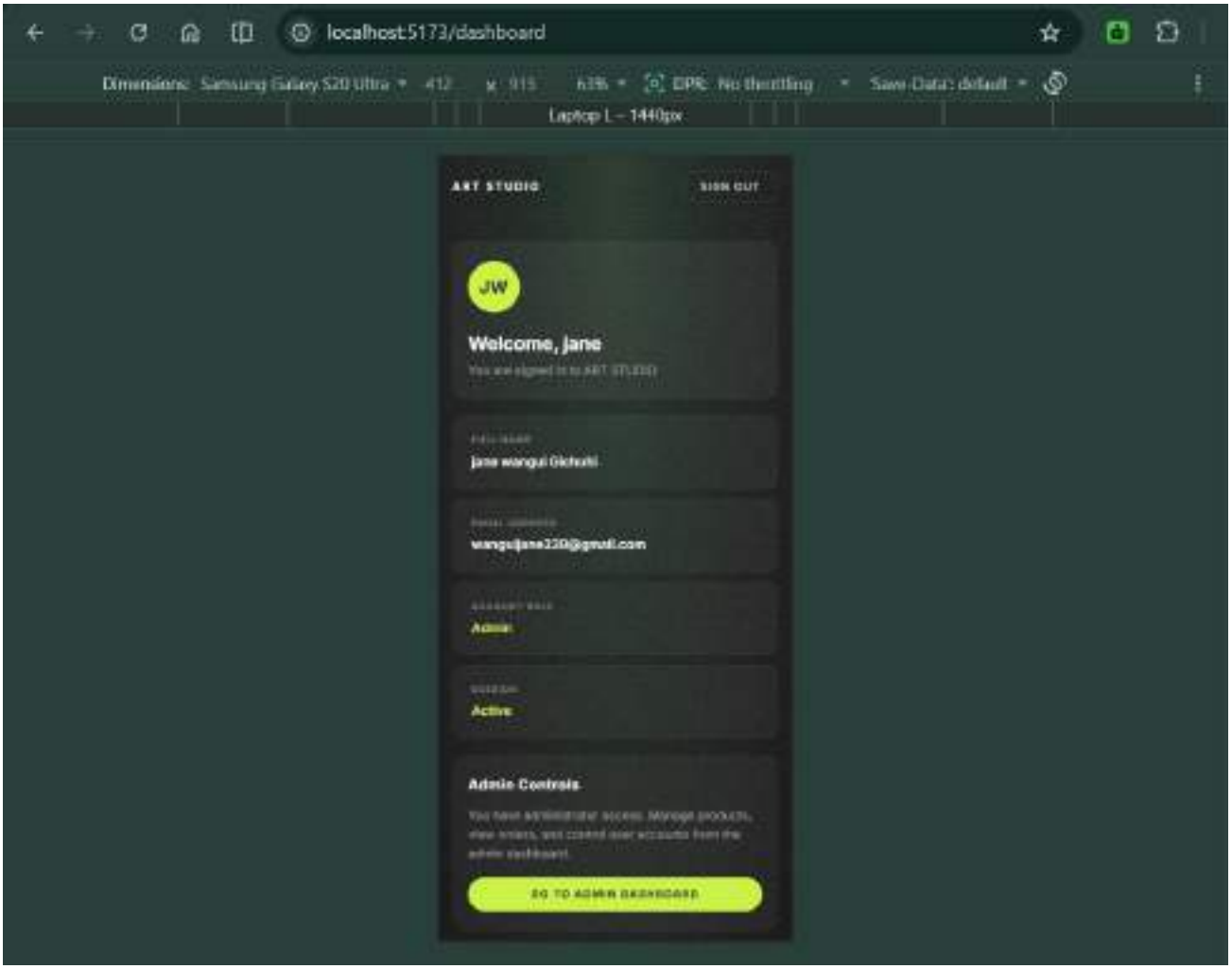

Fig 1: A mobile-responsive Art Studio user dashboard showing profile details, active session status, and admin access controls.

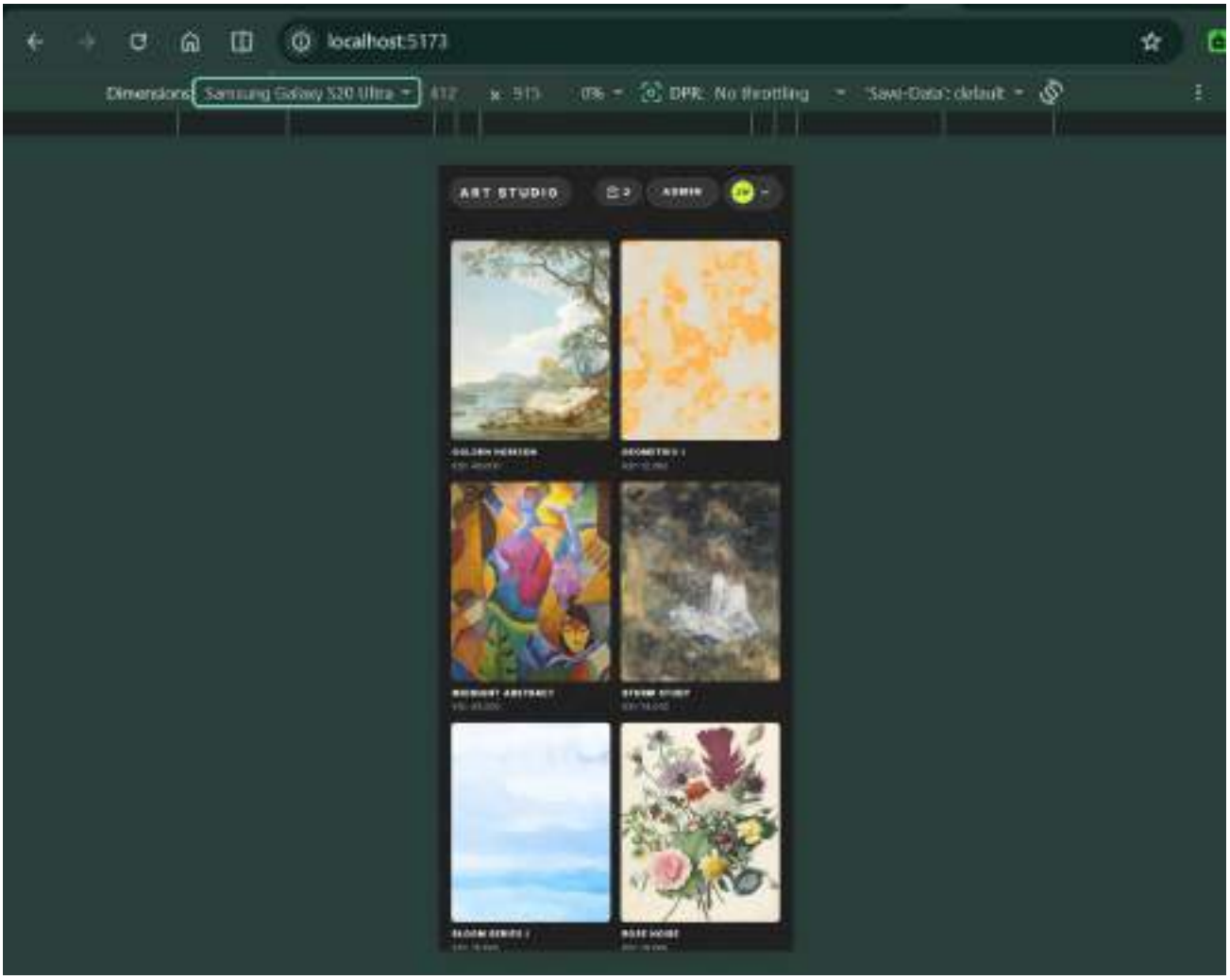

Fig 2: A mobile-responsive Art Studio gallery interface displaying artwork cards with images, titles, and prices.

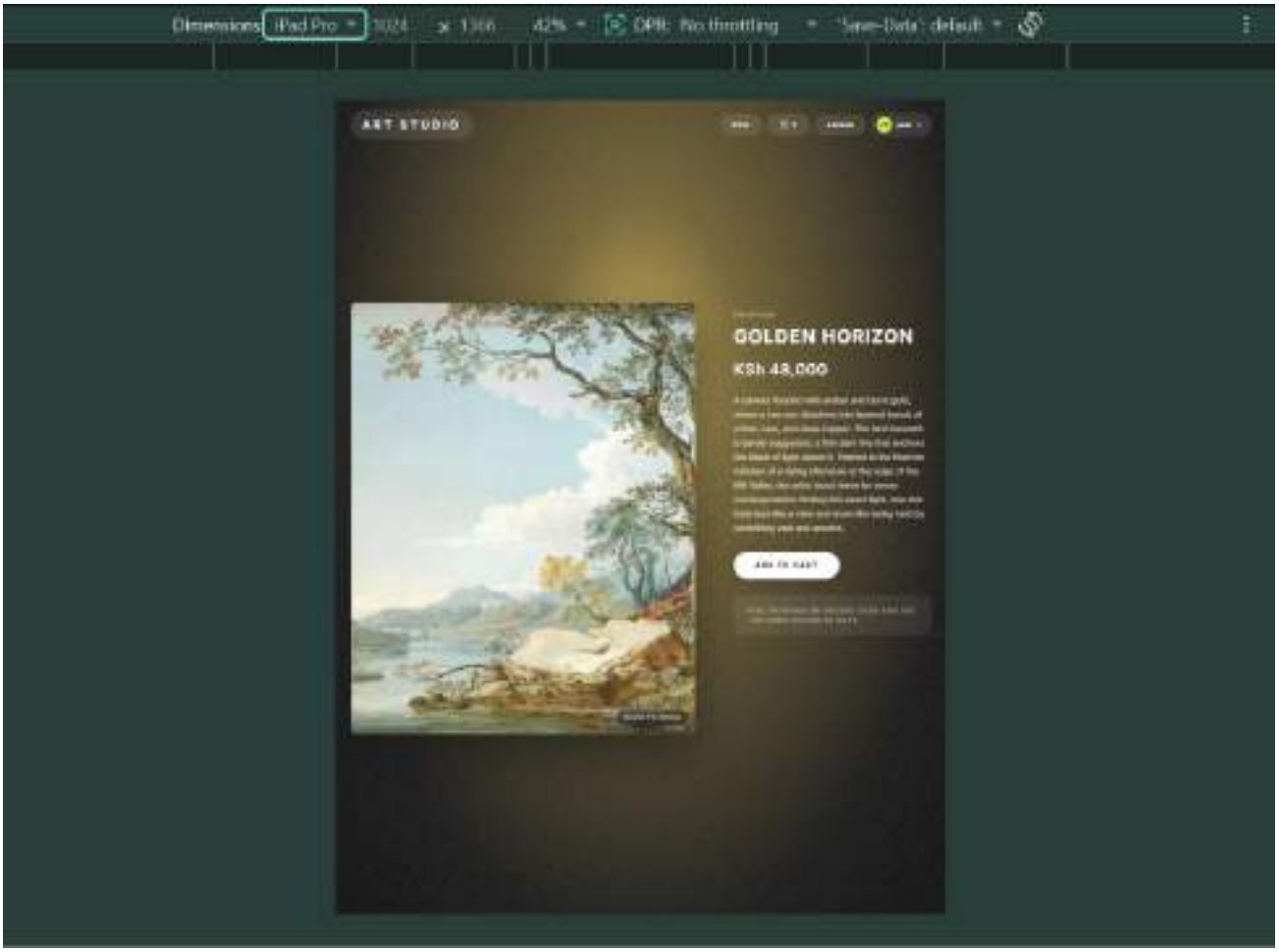

Fig 3: A tablet-responsive Art Studio product detail page showcasing the "Golden Horizon" artwork with its image, description, price, and add-to-cart button.

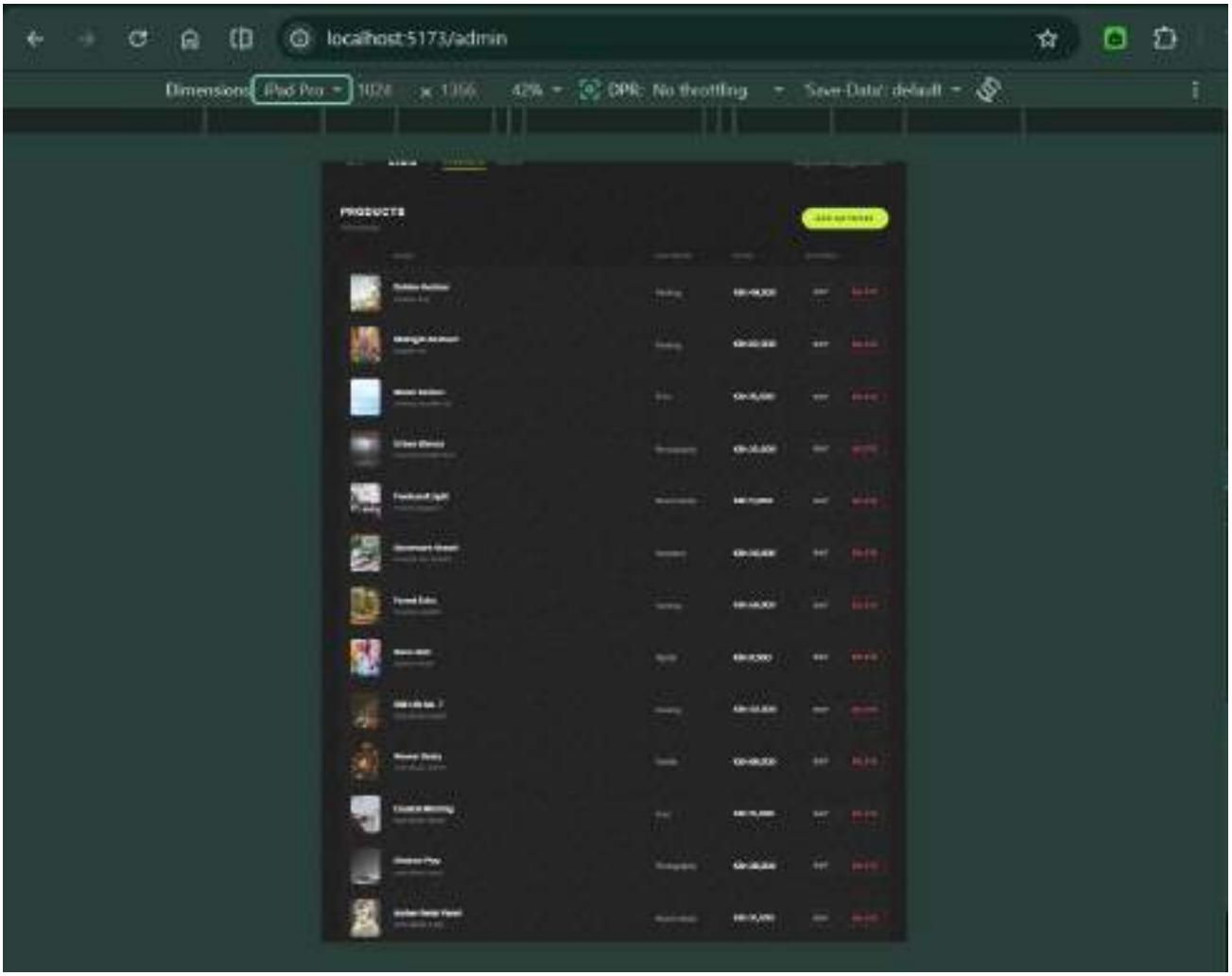

Fig 4: An iPad-responsive Art Studio admin products page displaying artwork inventory with categories, prices, and edit/delete management actions.

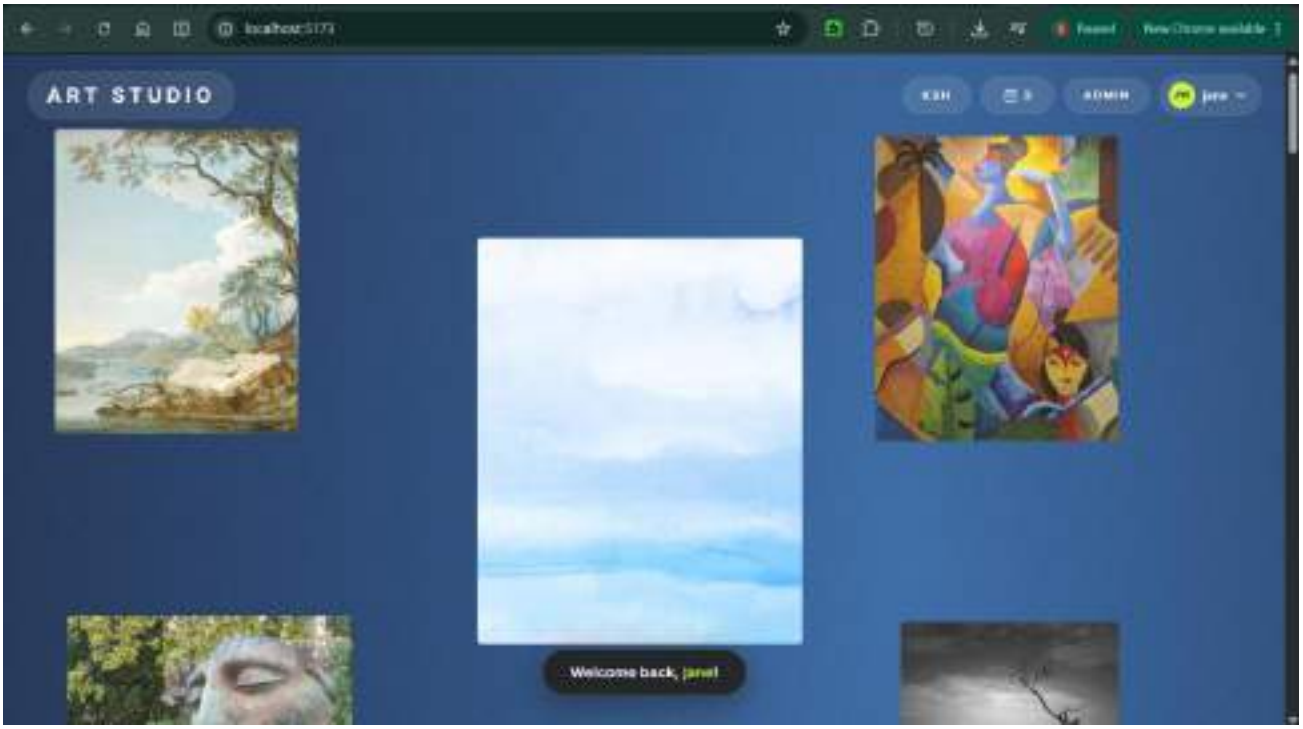

Fig 5: A desktop-responsive Art Studio homepage displaying a large gallery-style artwork layout with navigation controls, cart count, admin access, and user profile menu.

My reflection

Through this project, I learned how to design and test a responsive Art Studio web application across mobile, tablet, and desktop screens. The screenshots show that the system includes product browsing, artwork details, user dashboard, cart access, and admin controls. This helped me understand UI consistency, responsiveness, and user-friendly navigation.